# STAR RX72N - rx_spi_link Library

1.0.0

Generated by Doxygen 1.16.1

# Chapter 1

# Directory Hierarchy

## 1.1 Directories

# Chapter 2

# File Index

## 2.1 File List

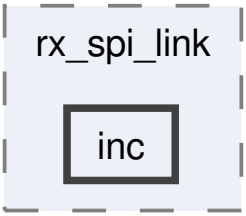
Here is a list of all files with brief descriptions:

# Chapter 3

# Directory Documentation

## 3.1 libs/rx_spi_link/inc Directory Reference

Directory dependency graph for inc:

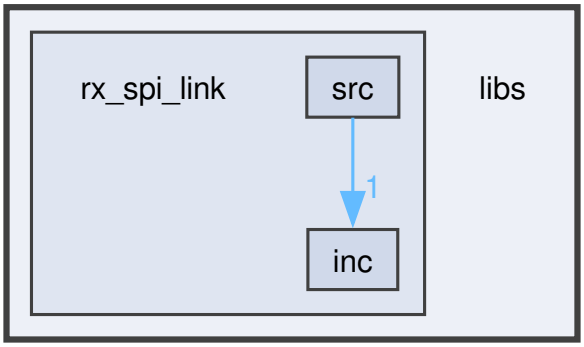



Files

- file rx_spi_link.h
  SPI Link Layer with HARQ.

## 3.2 libs Directory Reference

Directory dependency graph for libs:

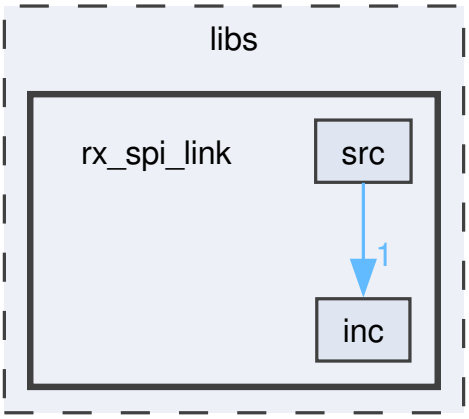


Directories

- directory rx_spi_link

## 3.3 libs/rx_spi_link Directory Reference

Directory dependency graph for rx_spi_link:

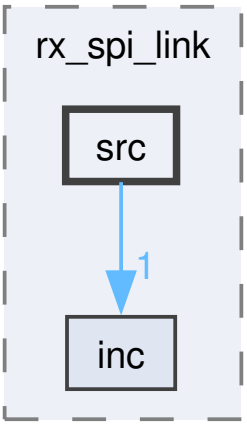



Directories

- directory inc
- directory src

## 3.4 libs/rx_spi_link/src Directory Reference

Directory dependency graph for src:




Files

- file rx_spi_link.c

  SPI Link Layer with HARQ Implementation.

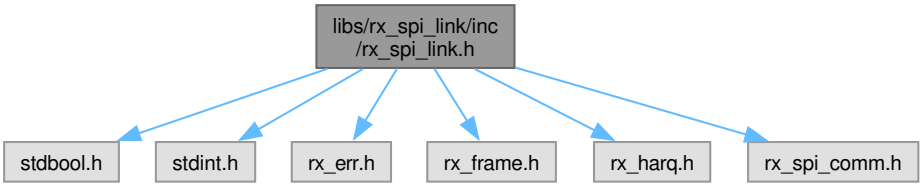
# Chapter 4

# File Documentation

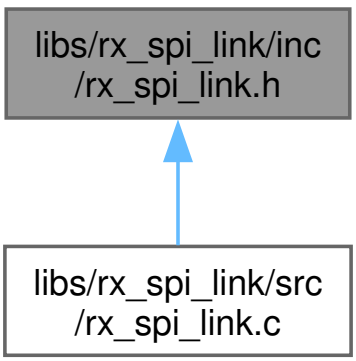
## 4.1 libs/rx_spi_link/inc/rx_spi_link.h File Reference

SPI Link Layer with HARQ.

#include <stdbool.h>
#include <stdint.h>
#include "rx_err.h"
#include "rx_frame.h"
#include "rx_harq.h"
#include "rx_spi_comm.h"
Include dependency graph for rx_spi_link.h:




This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_spi_link_config_t
  SPI link layer configuration.
- struct rx_spi_link_receive_result_t
  Result of a successful receive operation.
- struct rx_spi_link_t
  SPI link layer handle.

Enumerations

- enum rx_spi_link_constants_t : uint8_t { k_spi_link_default_max_retries , k_spi_link_default_fec_enabled }
  SPI link layer compile-time constants.
- enum rx_spi_link_timing_t : uint16_t { k_spi_link_ack_timeout_ms }
  SPI link layer timing constants.
- enum rx_spi_link_buffer_t : uint16_t { k_spi_link_max_encoded_payload }
  Buffer size constants for SPI link layer.
- enum rx_spi_link_state_t : uint8_t { k_spi_link_state_idle = 0 , k_spi_link_state_waiting_ack = 1 , k_spi_link_state_retransmitting = 2 , k_spi_link_state_error = 3 }
  SPI link layer state machine states.

Functions

- rx_err_t rx_spi_link_init (rx_spi_link_t *link, const rx_spi_link_config_t *config)
  Initialize SPI link layer.
- rx_err_t rx_spi_link_deinit (rx_spi_link_t *link)
  Deinitialize SPI link layer.
- rx_err_t rx_spi_link_send (rx_spi_link_t *link, rx_frame_type_t type, const uint8_t *payload, uint32_t payload_len)
  Send payload with HARQ reliability over SPI.
- rx_err_t rx_spi_link_receive (rx_spi_link_t *link, rx_spi_link_receive_result_t *result, uint32_t timeout_ms)
  Receive and decode a frame from SPI with HARQ.
- rx_err_t rx_spi_link_reset (rx_spi_link_t *link)
  Reset SPI link layer for new session.
- rx_spi_link_state_t rx_spi_link_get_state (const rx_spi_link_t *link)
  Get current SPI link state.
- bool rx_spi_link_fec_enabled (const rx_spi_link_t *link)
  Check if FEC is enabled on this link.

### 4.1.1 Detailed Description

SPI Link Layer with HARQ.

Implements a reliability layer between the communication manager and the raw SPI transport. This module mirrors the Go gateway's internal/link/spi.go, providing:

- FEC encoding on TX (convolutional K=7, rate 1/2)
- FEC decoding on RX (soft-decision Viterbi)
- Chase Combining across retransmissions (soft bit accumulation)
- ACK/NACK handshake with sequence tracking
- Configurable retries (default 3)

#### 4.1.1.1 Architecture

```
rx_comm_manager
     |
     v
+--------------+
| rx_spi_link  |  <-- THIS MODULE (HARQ)
|              |
| FEC encode   |  TX: payload -> FEC encode -> frame -> SPI
| FEC decode   |  RX: SPI -> frame -> soft bits -> Viterbi -> payload
| Chase Cmb    |  Retry: combine soft bits across attempts
| ACK/NACK     |  Handshake with gateway
+--------------+
     |
     v
rx_spi_comm (frame + CRC-32 + raw SPI)
```

#### 4.1.1.2 Protocol Flow (TX)

1. Application calls rx_spi_link_send(payload)
2. If FEC enabled: rx_harq_encode(payload) -> encoded_payload (2N+2 bytes)
3. rx_spi_comm_send(encoded_payload, flags=FEC_ENABLED|REQUIRES_ACK)
4. Wait for ACK/NACK from gateway (via rx_spi_comm_receive)
5. On NACK or timeout: retransmit (up to max_retries)
6. On ACK: return success

#### 4.1.1.3 Protocol Flow (RX)

1. rx_spi_comm_receive() returns a frame
2. If ACK/NACK frame: dispatch to pending TX operation
3. If data frame with FEC flag: convert payload bytes to soft bits
4. Pass soft bits to rx_harq_decode() (Chase Combining + Viterbi)
5. If decode succeeds: send ACK, return decoded payload
6. If decode fails and retries remain: store in combiner, send NACK
7. If max retries exceeded: return error

#### 4.1.1.4 Comparison: USB CDC vs SPI Link

| Feature | USB CDC (rx_usb_comm) | SPI Link (rx_spi_link) |
|---|---|---|
| FEC encoding | No | Yes (K=7, rate 1/2) |
| Viterbi decoding | No | Yes (soft-decision) |
| Chase Combining | No | Yes (int16 accumulation) |
| ACK/NACK | No (USB HW handles) | Yes (explicit handshake) |
| Retransmission | No (USB HW handles) | Yes (max 3 attempts) |
| CRC-32 | Yes | Yes (frame layer) |
| Sequence tracking | Yes (shared session) | Yes (shared session) |

#### 4.1.1.5 Memory Requirements

| Resource | Size | Notes |
|---|---|---|
| rx_spi_link_t | ~86 KB | Dominated by HARQ handle (82 KB) |
| FEC encode buffer | 2050 B | Max encoded payload (1024*2+2) |
| Soft bits buffer | 16400 B | Part of HARQ handle |
| Stack (send) | ~256 B | Function locals + encode buffer |
| Stack (receive) | ~128 B | Function locals |

NASA Power of 10 Compliance

- Rule 1: No goto, setjmp/longjmp, recursion
- Rule 2: All loops bounded (max_retries, k_fec_max_symbols)
- Rule 3: Zero dynamic allocation (all static buffers)
- Rule 4: Functions < 60 lines
- Rule 5: Minimum 2 pre/postconditions per function
- Rule 8: C23 typed enums for all constants
- Rule 9: Function pointers for dependency inversion (INTENTIONAL)
- Rule 10: Compiled with -Wall -Wextra -Werror

SOLID Principles

- S: Only handles HARQ reliability (not framing, not transport)
- O: Configurable via rx_spi_link_config_t without modifying code
- L: Substitutable for rx_spi_comm (same send/receive semantics)
- I: Small focused API (init, deinit, send, receive, reset)
- D: Depends on rx_spi_comm and rx_harq abstractions

See also

star-gateway/internal/link/spi.go Go reference implementation
rx_harq.h HARQ protocol and Chase Combiner
rx_fec.h FEC encoder/decoder
rx_spi_comm.h Raw SPI transport with framing

Author

Locked, Inc.

Date

2026-02-14

Copyright



Since

Version 1.0.0

Definition in file rx_spi_link.h.

### 4.1.2 Data Structure Documentation

#### 4.1.2.1 struct rx_spi_link_config_t

SPI link layer configuration.

Passed to rx_spi_link_init() to configure the link behavior. All zero values use defaults.

Invariant

spi_handle must be non-NULL and initialized via rx_spi_comm_init()

max_retries in range [0, 255], where 0 means use default (3)

fec_enabled is boolean (0 or 1)

Example

```
rx_spi_link_config_t cfg = {
    .spi_handle  = &g_spi_comm,
    .fec_enabled = true,
    .max_retries = 3,
};
```

See also

rx_spi_link_init() Initialization function consuming this config

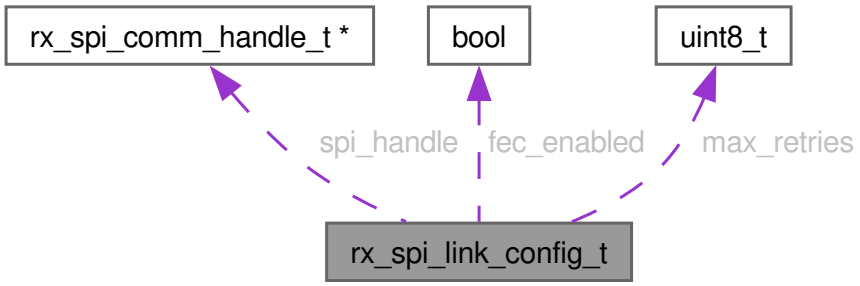
rx_spi_comm_handle_t Underlying SPI transport handle type

Since

Version 1.0.0

Definition at line 303 of file rx_spi_link.h.

Collaboration diagram for rx_spi_link_config_t:




Data Fields

| | | |
|---|---|---|
| bool | fec_enabled | Enable FEC encoding/decoding (true=HARQ with Chase Combining, false=raw frames) |

| | | |
|---|---|---|
| uint8_t | max_retries | Maximum transmission attempts (0=use default of 3, range [0-255]) |
| rx_spi_comm_handle_t * | spi_handle | Underlying SPI transport (REQUIRED, must be non-NULL and initialized via rx_spi_comm_init) |

#### 4.1.2.2 struct rx_spi_link_receive_result_t

Result of a successful receive operation.

Contains the decoded payload and metadata about the decoding process. Mirrors Go's harq.ReceiveResult.

Warning

This structure contains a large 1024-byte payload buffer (total struct size ~1032 bytes). MUST be statically allocated (global or static variable) only. DO NOT allocate on the stack (causes stack overflow when passing to rx_spi_link_receive()). DO NOT use dynamic allocation (malloc/new) - zero heap usage required per NASA Rule 3 and project coding guidelines. All buffers must use enum-defined sizes for compile-time allocation.

Invariant

payload_len <= k_harq_max_payload (1024 bytes)

sequence in range [0, 65535] (wraps around)

combining_count in range [0, max_retries], typically [1-4]

See also

rx_spi_link_receive() Function that populates this result structure

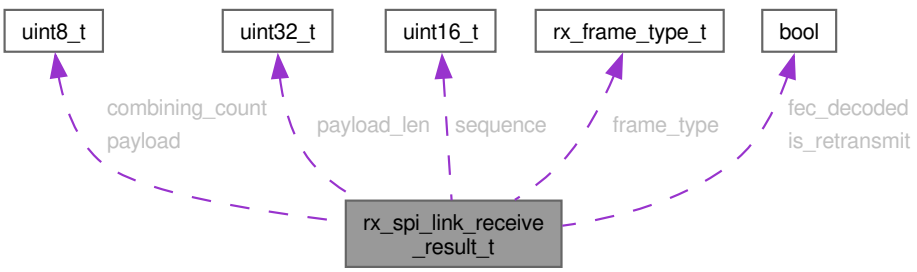
star-gateway/internal/harq/harq.go Go equivalent (ReceiveResult)

Since

Version 1.0.0

Definition at line 333 of file rx_spi_link.h.

Collaboration diagram for rx_spi_link_receive_result_t:



Data Fields

| | | |
|---|---|---|
| uint8_t | combining_count | Number of Chase Combining attempts used (1 = first attempt, >1 = retransmissions combined) |
| bool | fec_decoded | True if FEC/Viterbi decoding was applied, false if raw passthrough |
| rx_frame_type_t | frame_type | Frame type (e.g., k_frame_type_command, k_frame_type_telemetry) |
| bool | is_retransmit | True if frame had k_frame_flag_retransmit set, false for first transmission |
| uint8_t | payload[k_harq_max_payload] | Decoded payload data (buffer size: k_harq_max_payload = 1024 bytes) |
| uint32_t | payload_len | Decoded payload length in bytes (range: [0, k_harq_max_payload]) |
| uint16_t | sequence | Frame sequence number (range: [0, 65535], wraps around) |

#### 4.1.2.3 struct rx_spi_link_t

SPI link layer handle.

Contains the HARQ state machine, FEC encoder/decoder, Chase Combiner, and a pointer to the underlying SPI transport. Approximately 86 KB total (dominated by the HARQ handle's FEC decoder buffers).

#### 4.1.2.4 Memory Layout

| Field | Size | Description |
|---|---|---|
| spi_handle | 8 B | Pointer to SPI transport |
| harq | ∼82 KB | HARQ handle (FEC + Chase Combiner) |
| state | 1 B | Current link state |
| fec_enabled | 1 B | FEC enable flag |
| max_retries | 1 B | Max transmission attempts |
| initialized | 1 B | Initialization flag |
| fec_buf | 2050 B | FEC encode output buffer |

Invariant

When initialized == true, spi_handle must be non-NULL

state must be one of k_spi_link_state_∗ values

max_retries in range [1, 255] when initialized (0 converted to 3 at init)

fec_encode_buf size == k_spi_link_max_encoded_payload (2050 bytes)

Warning

This structure is ~86 KB. Allocate statically, not on the stack.

See also

rx_spi_link_init() Initialization function

rx_spi_link_config_t Configuration structure

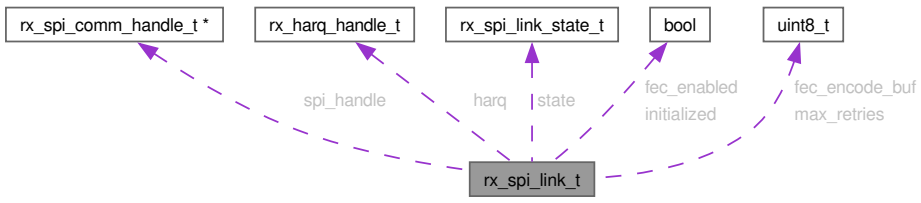
rx_harq_handle_t HARQ handle containing FEC encoder/decoder

Since

Version 1.0.0

Definition at line 380 of file rx_spi_link.h.

Collaboration diagram for rx_spi_link_t:




Data Fields

| | | |
|---|---|---|
| bool | fec_enabled | FEC enabled flag (true=HARQ, false=raw frames, immutable after init) |

| | | |
|---|---|---|
| uint8_t | fec_encode_buf[k_spi_link_max_encoded_payload] | Staging buffer for FEC-encoded payloads (size: 2049 bytes = 2∗1024+1). Used by rx_spi_link_send() to hold FEC-encoded data before passing to rx_spi_comm_send(). Formula: 2 ∗ k_harq_max_payload + ceiling(k_fec_tail_bits / 8), where k_fec_tail_bits = 6. |
| rx_harq_handle_t | harq | HARQ handle containing FEC encoder/decoder and Chase Combiner (∼82 KB) |
| bool | initialized | Initialization flag (true=ready for use, false=must call rx_spi_link_init) |
| uint8_t | max_retries | Maximum transmission attempts (range [1-255], 0 at init converted to 3) |
| rx_spi_comm_handle_t ∗ | spi_handle | Underlying SPI transport (must be non-NULL when initialized, set via config) |
| rx_spi_link_state_t | state | Current link state (idle, waiting_ack, retransmitting, error) |

### 4.1.3 Enumeration Type Documentation

#### 4.1.3.1 rx_spi_link_buffer_t

enum rx_spi_link_buffer_t : uint16_t

Buffer size constants for SPI link layer.

Defines buffer sizes for FEC-encoded payloads. Sized to accommodate the maximum payload after FEC encoding (rate 1/2) plus tail bits.

Formula Derivation:

- FEC rate 1/2: Each input byte produces 2 output bytes
- k_harq_max_payload = 1024 bytes
- FEC encoded size = 2 ∗ k_harq_max_payload = 2 ∗ 1024 = 2048 bytes
- k_fec_tail_bits = 6 bits (trellis termination, K=7 constraint length)
- Tail bytes = ceiling(k_fec_tail_bits / 8) = ceiling(6 / 8) = 1 byte
- Total = 2048 + 1 = 2049 bytes

See also

rx_harq.h HARQ protocol maximum payload size

rx_fec.h FEC encoding parameters (rate 1/2, tail bits)

Since

Version 1.0.0

Enumerator

| | |
|---|---|
| k_spi_link_max_encoded_payload | Maximum FEC-encoded payload size in bytes. Formula: bytes = (2 ∗ k_harq_max_payload) + ceiling(k_fec_tail_bits / 8) = (2 ∗ 1024) + ceiling(6 / 8) = 2048 + 1 = 2049. FEC rate 1/2 doubles the payload size, and tail bits (6 bits = 1 byte after ceiling) are appended for trellis termination. |

Definition at line 204 of file rx_spi_link.h.

```
00204             : uint16_t {
00205   k_spi_link_max_encoded_payload =
00206     2049, /**< Maximum FEC-encoded payload size in bytes. Formula: bytes = (2 * k_harq_max_payload) +
      ceiling(k_fec_tail_bits / 8) = (2 * 1024) + ceiling(6 / 8) = 2048 + 1 = 2049. FEC rate 1/2 doubles the payload size, and
      tail bits (6 bits = 1 byte after ceiling) are appended for trellis termination. */
00207 } rx_spi_link_buffer_t;
```

#### 4.1.3.2 rx_spi_link_constants_t

enum rx_spi_link_constants_t : uint8_t

SPI link layer compile-time constants.

Protocol timing and retry parameters matching the Go gateway's internal/link/spi.go constants.

Invariant

max_retries must be in range [0, 255], where 0 means use default value of 3

fec_enabled is boolean (0 or 1)

```
// Example: Using default constants
rx_spi_link_config_t config = {
    .spi_handle = &spi_handle,
    .fec_enabled = k_spi_link_default_fec_enabled,  // 1 (enabled)
    .max_retries = k_spi_link_default_max_retries,  // 3 attempts
};
```

See also

star-gateway/internal/link/spi.go Go reference constants (maxRetries=3)

rx_spi_link_config_t Configuration structure using these defaults

Since

Version 1.0.0

Enumerator

| | |
|---|---|
| k_spi_link_default_max_retries | Default maximum transmission attempts (matches Go maxRetries=3). After 3 failed attempts (including Chase Combining), the link returns an error and the comm_manager may fail over to USB. |
| k_spi_link_default_fec_enabled | Default FEC enabled state (enabled by default, aligns with Go HARQ). FEC/HARQ is enabled by default for reliability. Can be disabled at runtime via config or test modes for raw SPI operation. |

Definition at line 148 of file rx_spi_link.h.

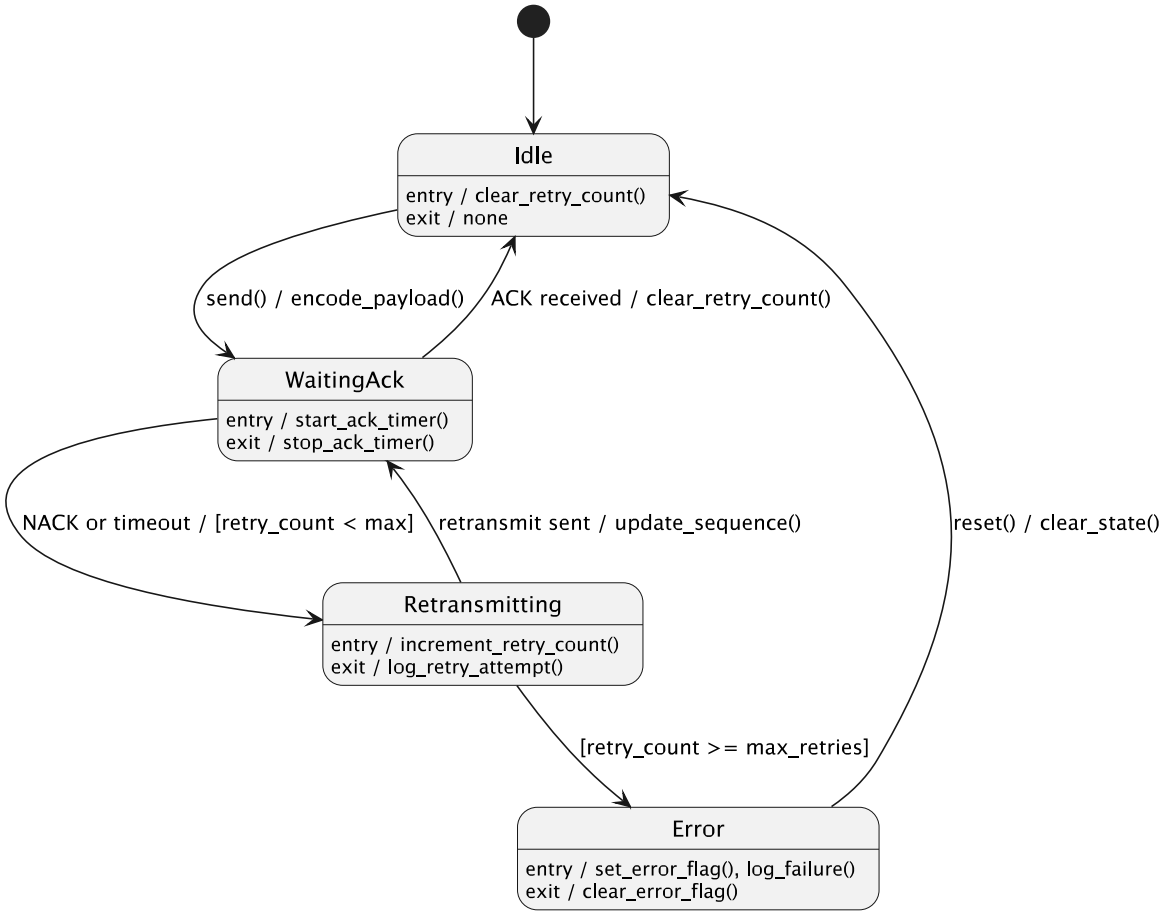
```
00148             : uint8_t {
00149     k_spi_link_default_max_retries =
00150         3, /**< Default maximum transmission attempts (matches Go maxRetries=3). After 3 failed attempts (including Chase
          Combining), the link returns an error and the comm_manager may fail over to USB. */
00151
00152     k_spi_link_default_fec_enabled =
00153         1, /**< Default FEC enabled state (enabled by default, aligns with Go HARQ). FEC/HARQ is enabled by default for
          reliability. Can be disabled at runtime via config or test modes for raw SPI operation. */
00154 } rx_spi_link_constants_t;
```

#### 4.1.3.3 rx_spi_link_state_t

enum rx_spi_link_state_t : uint8_t

SPI link layer state machine states.

Mirrors the Go gateway's harq.State enum. Tracks whether the link is idle, waiting for acknowledgment, retransmitting, or in error.




State Transition Table

| From State | To State | Event/Condition | Guard/Action | Entry Action | Exit Action |
|---|---|---|---|---|---|
| Idle | WaitingAck | send() called | encode_payload() | start_ack_timer() | none |
| WaitingAck | Idle | ACK received | clear_retry_count() | clear_retry_count() | stop_ack_timer() |
| WaitingAck | Retransmitting | NACK or timeout | retry_count < max_retries | increment_retry_count() | stop_ack_timer() |
| Retransmitting | WaitingAck | retransmit sent | update_sequence() | start_ack_timer() | log_retry_attempt() |

| From State | To State | Event/Condition | Guard/Action | Entry Action | Exit Action |
|---|---|---|---|---|---|
| Retransmitting | Error | max retries exceeded | retry_count >= max_retries | set_error_flag(), log() | log_retry_attempt() |
| Error | Idle | reset() called | clear_state() | clear_retry_counter() | clear_error_flag() |

See also

rx_spi_link_get_state() Query current state

rx_spi_link_reset() Reset to Idle state

Since

Version 1.0.0

Enumerator

| | |
|---|---|
| k_spi_link_state_idle | Ready for new send/receive |
| k_spi_link_state_waiting_ack | Frame sent, waiting for ACK/NACK |
| k_spi_link_state_retransmitting | Retransmitting after NACK/timeout |
| k_spi_link_state_error | Unrecoverable error (max retries) |

Definition at line 267 of file rx_spi_link.h.

```
00267             : uint8_t {
00268   k_spi_link_state_idle          = 0, /**< Ready for new send/receive */
00269   k_spi_link_state_waiting_ack   = 1, /**< Frame sent, waiting for ACK/NACK */
00270   k_spi_link_state_retransmitting = 2, /**< Retransmitting after NACK/timeout */
00271   k_spi_link_state_error         = 3, /**< Unrecoverable error (max retries) */
00272 } rx_spi_link_state_t;
```

#### 4.1.3.4 rx_spi_link_timing_t

enum rx_spi_link_timing_t : uint16_t

SPI link layer timing constants.

Timeout values for ACK waiting and buffer sizing. Matches Go gateway's ackTimeout = 10ms and FEC encoding buffer requirements.

Invariant

ack_timeout_ms must be > 0 and < 65535 milliseconds

max_encoded_payload = 2 * k_harq_max_payload + k_fec_tail_bits (2050 bytes for 1024-byte max payload)

```
// Example: Using timing constants for ACK wait
rx_err_t err = rx_spi_comm_receive(spi_handle, &ack_frame, k_spi_link_ack_timeout_ms);
if (err == k_rx_err_timeout) {
    // No ACK received within 10ms, retry transmission
}
```

See also

star-gateway/internal/link/spi.go Go ackTimeout constant (10ms)

rx_fec.h FEC encoding parameters (rate 1/2, tail bits)

Since

Version 1.0.0

Enumerator

| | |
|---|---|
| k_spi_link_ack_timeout_ms | ACK/NACK wait timeout in milliseconds (matches Go ackTimeout=10ms). How long to wait for ACK/NACK after sending a frame. If no response arrives within this window, the frame is retransmitted. |

Definition at line 179 of file rx_spi_link.h.

```
00179             : uint16_t {
00180   k_spi_link_ack_timeout_ms =
00181     10, /**< ACK/NACK wait timeout in milliseconds (matches Go ackTimeout=10ms). How long to wait for ACK/NACK
        after sending a frame. If no response arrives within this window, the frame is retransmitted. */
00182 } rx_spi_link_timing_t;
```

### 4.1.4 Function Documentation

#### 4.1.4.1 rx_spi_link_deinit()

```
rx_err_t rx_spi_link_deinit (
                rx_spi_link_t * link)   [nodiscard]
```

Deinitialize SPI link layer.

Deinitializes the HARQ handle and marks the link as uninitialized. Does NOT deinitialize the underlying SPI transport.

Parameters

| | | |
|---|---|---|
| in,out | link | Link handle to deinitialize |

Returns

rx_err_t Error code

Return values

| | |
|---|---|
| k_rx_ok | Success |
| k_rx_err_invalid_arg | link isnullptr |
| k_rx_err_invalid_state | link not initialized |

Precondition

link must be non-NULL

link must be initialized (link->initialized == true)

Postcondition

link->initialized == false

link->harq internal state deinitialized (HARQ handle invalid)

Note

Does NOT deinitialize the underlying SPI transport handle

See also

rx_spi_link_init() Initialization counterpart

rx_harq_deinit() Internal HARQ deinitialization

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

Definition at line 349 of file rx_spi_link.c.

```
00350 {
00351   /* Pre-condition 1 */
00352   if (link == nullptr) {
00353     return k_rx_err_invalid_arg;
00354   }
00355
00356   /* Pre-condition 2 */
00357   if (!link->initialized) {
00358     return k_rx_err_invalid_state;
00359   }
00360
00361   /* Deinit HARQ subsystem. HARQ deinit only fails if FEC encoder/decoder
00362    * deinit fails, which cannot happen after a successful init on target. */
00363   (void)(rx_harq_deinit(&link->harq));
00364
00365   /* Post-condition: Mark uninitialized */
00366   link->initialized = false;
00367   link->state       = k_spi_link_state_idle;
00368
00369   rx_log_info(s_tag, "Deinitialized");
00370   return k_rx_ok;
00371 }
```

References rx_spi_link_t::harq, rx_spi_link_t::initialized, k_spi_link_state_idle, s_tag, and rx_spi_link_t::state.

#### 4.1.4.2 rx_spi_link_fec_enabled()

```
bool rx_spi_link_fec_enabled (
        const rx_spi_link_t * link)
```

Check if FEC is enabled on this link.

Queries the FEC/HARQ enabled flag. Returns false if link is NULL. Does not modify link state. FEC setting is configured at init time via rx_spi_link_config_t.fec_enabled and cannot be changed at runtime.

Parameters

| | | |
|---|---|---|
| in | link | Link handle (may be NULL) |

Returns

bool FEC enabled status

Return values

| | |
|---|---|
| true | FEC encoding/decoding is active (Chase Combining, Viterbi) |
| false | FEC disabled (raw payload transmission) or link isnullptr |

Precondition

link may be NULL (function handles gracefully by returning false)

If link is non-NULL, may be in any state (initialized or not)

Postcondition

No state change (pure query function)

Return value reflects FEC setting at time of call (constant for link lifetime)

Note

Thread-safe: Read-only operation, no synchronization needed

NULL-safe: Returns false when link isnullptr

FEC setting is immutable after initialization (cannot be changed at runtime)

When FEC is enabled, rx_spi_link_send/receive use HARQ with Chase Combining

See also

rx_spi_link_config_t Configuration structure with fec_enabled field

rx_spi_link_init() Initialization function that sets FEC mode

k_spi_link_default_fec_enabled Default FEC setting (enabled = 1)

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

Definition at line 829 of file rx_spi_link.c.

```
00830 {
00831   if (link == nullptr || !link->initialized) {
00832     return false;
00833   }
00834   return link->fec_enabled;
00835 }
```

References rx_spi_link_t::fec_enabled, and rx_spi_link_t::initialized.

#### 4.1.4.3 rx_spi_link_get_state()

```
rx_spi_link_state_t rx_spi_link_get_state (
                const rx_spi_link_t * link)
```

Get current SPI link state.

Returns the current state of the SPI link state machine. Safe to call with nullptr (returns error state). Does not modify link state.

Parameters

| | | |
|---|---|---|
| in | link | Link handle (may be NULL) |

Returns

rx_spi_link_state_t Current link state

Return values

| | |
|---|---|
| k_spi_link_state_idle | Link is ready for new send/receive |
| k_spi_link_state_waiting_ack | Waiting for ACK/NACK response |
| k_spi_link_state_retransmitting | Retransmitting after NACK/timeout |
| k_spi_link_state_error | Unrecoverable error (also returned if link is NULL) |

Precondition

link may be NULL (function handles gracefully)

If link is non-NULL, may be in any state (initialized or not)

Postcondition

No state change (pure query function)

Return value reflects state at time of call (may change immediately after)

Note

Thread-safe: Read-only operation, no synchronization needed

NULL-safe: Returns k_spi_link_state_error when link isnullptr

Does not distinguish between ”link is NULL” vs ”link is in error state”

See also

rx_spi_link_t Link handle structure

rx_spi_link_state_t State enumeration with all possible values

rx_spi_link_reset() Reset link to idle state

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

Definition at line 817 of file rx_spi_link.c.

```
00818 {
00819   if (link == nullptr || !link->initialized) {
00820     return k_spi_link_state_error;
00821   }
00822   return link->state;
00823 }
```

References rx_spi_link_t::initialized, k_spi_link_state_error, and rx_spi_link_t::state.

#### 4.1.4.4 rx_spi_link_init()

```
rx_err_t rx_spi_link_init (
                rx_spi_link_t * link,
                const rx_spi_link_config_t * config)   [nodiscard]
```

Initialize SPI link layer.

Initializes the HARQ handle (including FEC encoder/decoder and Chase Combiner) and stores configuration. The underlying SPI transport must already be initialized.

Parameters

| | | |
|---|---|---|
| out | link | Link handle to initialize |
| in | config | Configuration (spi_handle is REQUIRED) |

Returns

rx_err_t Error code

Return values

| | |
|---|---|
| k_rx_ok | Success |
| k_rx_err_invalid_arg | link or config isnullptr; config->spi_handle isnullptr |

Precondition

link must point to valid, allocated memory (~86 KB)

config->spi_handle must be initialized via rx_spi_comm_init()

Postcondition

link->initialized == true

link->harq initialized with FEC if fec_enabled

Note

Thread Safety: Not thread-safe. Call from single thread during init.

See also

rx_spi_link_deinit() Cleanup counterpart

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

Definition at line 300 of file rx_spi_link.c.

```
00301 {
00302   /* Pre-condition 1: NULL checks */
00303   if (link == nullptr || config == nullptr) {
00304     return k_rx_err_invalid_arg;
00305   }
00306
00307   /* Pre-condition 2: SPI handle required */
00308   if (config->spi_handle == nullptr) {
00309     rx_log_error(s_tag, "SPI handle is required");
00310     return k_rx_err_invalid_arg;
00311   }
00312
00313   *link = s_zero_link;
00314
00315   /* Store configuration */
00316   link->spi_handle  = config->spi_handle;
00317   link->fec_enabled = config->fec_enabled;
00318   link->max_retries =
00319     (config->max_retries > 0) ? config->max_retries : k_spi_link_default_max_retries;
00320
00321   /* Initialize HARQ handle (includes FEC encoder/decoder and Chase Combiner) */
00322   const rx_harq_config_t harq_cfg = {
00323     .max_retries  = link->max_retries,
```

```
00324     .fec_enabled  = (int)link->fec_enabled ? 1U : 0U,
00325     .max_combines = link->max_retries,
00326   };
00327
00328   /* HARQ init only fails if FEC encoder/decoder init fails, which cannot
00329    * happen with valid config on the target. This is a defensive assertion. */
00330   (void)(rx_harq_init(&link->harq, &harq_cfg));
00331
00332   /* Post-condition 1: Mark initialized */
00333   link->state       = k_spi_link_state_idle;
00334   link->initialized = true;
00335
00336   /* Post-condition 2: Verify HARQ is ready.
00337    * rx_harq_get_state returns idle after successful init; this is a
00338    * defensive invariant check. */
00339
00340   rx_log_info(s_tag, "Init OK");
00341
00342   return k_rx_ok;
00343 }
```

References rx_spi_link_config_t::fec_enabled, rx_spi_link_t::fec_enabled, rx_spi_link_t::harq, rx_spi_link_t::initialized, k_spi_link_default_max_retries, k_spi_link_state_idle, rx_spi_link_config_t::max_retrie rx_spi_link_t::max_retries, s_tag, s_zero_link, rx_spi_link_config_t::spi_handle, rx_spi_link_t::spi_handle, and rx_spi_link_t::state.

#### 4.1.4.5 rx_spi_link_receive()

```
rx_err_t rx_spi_link_receive (
                rx_spi_link_t * link,
                rx_spi_link_receive_result_t * result,
                uint32_t timeout_ms)   [nodiscard]
```

Receive and decode a frame from SPI with HARQ.

Receives a frame from the SPI transport, handles ACK/NACK dispatch, and performs FEC decoding with Chase Combining if applicable.

#### 4.1.4.6 Algorithm

1. Call rx_spi_comm_receive() with timeout
2. If frame is ACK/NACK: handle internally (dispatch), return no-data
3. If frame is PING: auto-respond with PONG (handled by spi_comm)
4. If data frame with FEC flag: a. Convert payload bytes to soft bits (byte -> 8 soft bits) b. If retransmit flag: add to Chase Combiner, decode combined c. If first attempt: decode directly, store in combiner on failure d. On decode success: send ACK, populate result e. On decode failure: send NACK, return error
5. If data frame without FEC: pass payload through directly, send ACK

Parameters

| | | |
|---|---|---|
| in,out | link | Initialized link handle |
| out | result | Decoded payload and metadata |
| in | timeout_ms | Receive timeout (0 = non-blocking) |

Returns

rx_err_t Error code

Return values

| | |
|---|---|
| k_rx_ok | Frame received and decoded successfully |
| k_rx_err_invalid_arg | link or result isnullptr |
| k_rx_err_invalid_state | link not initialized |
| k_rx_err_timeout | No frame received within timeout |
| k_rx_err_protocol_error | FEC decode failed (NACK sent) |
| k_rx_err_crc_mismatch | CRC validation failed at frame layer |

Precondition

link must be initialized

result must point to valid memory

Postcondition

On success: result populated with decoded payload and metadata

On ACK/NACK frame: returns k_rx_err_timeout (control frame, not data)

Note

Thread Safety: Not thread-safe. Dedicate one thread to receiving.

See also

rx_spi_link_send() Send path

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

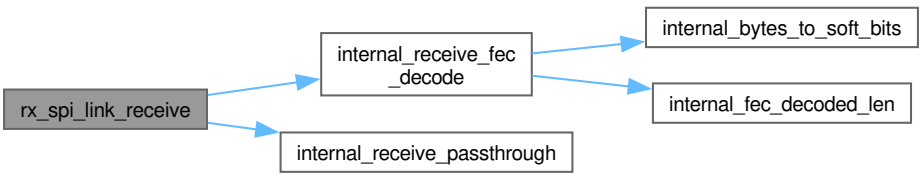
Definition at line 726 of file rx_spi_link.c.

```
00727 {
00728   /* Pre-condition 1: NULL checks */
00729   if (link == nullptr || result == nullptr) {
00730     return k_rx_err_invalid_arg;
00731   }
00732
00733   /* Pre-condition 2: State check */
00734   if (!link->initialized) {
00735     return k_rx_err_invalid_state;
00736   }
00737
00738   /* Receive raw frame from SPI transport */
00739   rx_frame_t frame = {};
00740   *result          = (rx_spi_link_receive_result_t){};
00741
```

```
00742   const rx_err_t recv_err = rx_spi_comm_receive(link->spi_handle, &frame, timeout_ms);
00743   if (recv_err != k_rx_ok) {
00744     return recv_err; /* Timeout or error */
00745   }
00746
00747   /* Handle ACK/NACK control frames internally (not data for application) */
00748   if (frame.header.type == k_frame_type_ack || frame.header.type == k_frame_type_nack) {
00749     /* Control frame - not application data */
00750     return k_rx_err_no_data; /* Signal "no data" to caller */
00751   }
00752
00753   /* Handle PING/PONG/RESET (already handled by rx_spi_comm internally) */
00754   const bool is_ping      = (bool)(frame.header.type == k_frame_type_ping);
00755   const bool is_pong      = (bool)(frame.header.type == k_frame_type_pong);
00756   const bool is_reset     = (bool)(frame.header.type == k_frame_type_reset);
00757   const bool is_reset_ack = (bool)(frame.header.type == k_frame_type_reset_ack);
00758   if ((bool)((int)is_ping | (int)is_pong | (int)is_reset | (int)is_reset_ack)) {
00759     return k_rx_err_no_data; /* Not application data */
00760   }
00761
00762   /* Data frame received - populate common metadata */
00763   result->sequence      = frame.header.sequence;
00764   result->frame_type    = frame.header.type;
00765   result->is_retransmit = (bool)((frame.header.flags & k_frame_flag_retransmit) != 0);
00766
00767   /* Check if FEC decoding is needed */
00768   const bool has_fec_flag = (bool)((frame.header.flags & k_frame_flag_fec_enabled) != 0);
00769
00770   const bool fec_enabled_flag = link->fec_enabled;
00771   const bool fec_frame_flag   = has_fec_flag;
00772   const bool has_payload      = (bool)(frame.header.length > 0);
00773   if ((bool)((int)fec_enabled_flag & (int)fec_frame_flag & (int)has_payload)) {
00774     /* FEC decode path: HARQ with Chase Combining */
00775     return internal_receive_fec_decode(link, &frame, result);
00776   }
00777
00778   /* No FEC: passthrough (copy payload directly) */
00779   return internal_receive_passthrough(link, &frame, result);
00780 }
```

References rx_spi_link_t::fec_enabled, rx_spi_link_receive_result_t::frame_type, rx_spi_link_t::initialized, internal_receive_fec_decode(), internal_receive_passthrough(), rx_spi_link_receive_result_t::is_retransmit, rx_spi_link_receive_result_t::sequence, and rx_spi_link_t::spi_handle.

Here is the call graph for this function:




#### 4.1.4.7 rx_spi_link_reset()

```
rx_err_t rx_spi_link_reset (
                rx_spi_link_t * link)   [nodiscard]
```

Reset SPI link layer for new session.

Resets the HARQ state machine, clears the Chase Combiner, and returns the link to idle state. Does NOT reset the shared session state (sequence numbers are managed by rx_session).

Parameters

| | | |
|---|---|---|
| in,out | link | Initialized link handle |

Returns

rx_err_t Error code

Return values

| | |
|---|---|
| k_rx_ok | Success |
| k_rx_err_invalid_arg | link isnullptr |
| k_rx_err_invalid_state | link not initialized |

Precondition

link must be non-NULL

link must be initialized (link->initialized == true)

Postcondition

link->state == k_spi_link_state_idle

Chase Combiner buffers cleared (all soft-decision history discarded)

Note

Does NOT reset TX/RX sequence numbers (managed externally by rx_session)

Safe to call after error state to recover link operation

See also

rx_harq_reset() Internal HARQ reset

rx_spi_link_get_state() Query current state

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

Definition at line 790 of file rx_spi_link.c.

```
00791 {
00792   /* Pre-condition 1 */
00793   if (link == nullptr) {
00794     return k_rx_err_invalid_arg;
00795   }
00796
00797   /* Pre-condition 2 */
00798   if (!link->initialized) {
00799     return k_rx_err_invalid_state;
00800   }
00801
00802   /* Reset HARQ (clears combiner and retry count). Only fails if harq is
00803    * null/uninitialized -- both prevented by link->initialized check above. */
00804   (void)(rx_harq_reset(&link->harq));
00805
00806   /* Post-condition: return to idle */
00807   link->state = k_spi_link_state_idle;
00808
00809   rx_log_debug(s_tag, "Link reset");
00810   return k_rx_ok;
00811 }
```

References rx_spi_link_t::harq, rx_spi_link_t::initialized, k_spi_link_state_idle, s_tag, and rx_spi_link_t::state.

#### 4.1.4.8 rx_spi_link_send()

```
rx_err_t rx_spi_link_send (
                rx_spi_link_t * link,
                rx_frame_type_t type,
                const uint8_t * payload,
                uint32_t payload_len)   [nodiscard]
```

Send payload with HARQ reliability over SPI.

Encodes payload with FEC (if enabled), sends via SPI transport, and waits for ACK/NACK. On NACK or timeout, retransmits up to max_retries.

#### 4.1.4.9 Algorithm

1. Validate parameters
2. If FEC enabled: rx_harq_encode(payload) -> encoded (2N+2 bytes)
3. For attempt = 1 to max_retries: a. Build frame with REQUIRES_ACK | FEC_ENABLED flags b. If retransmit: add RETRANSMIT flag c. Send via rx_spi_comm_send() d. Wait for ACK/NACK (rx_spi_comm_receive with ack_timeout_ms) e. On ACK: return success f. On NACK/timeout: continue to next attempt
4. All retries exhausted: return error

Parameters

| | | |
|---|---|---|
| in,out | link | Initialized link handle |
| in | type | Frame type (k_frame_type_command, etc.) |
| in | payload | Data to send |
| in | payload_len | Data length (0-1024 bytes) |

Returns

rx_err_t Error code

Return values

| | |
|---|---|
| k_rx_ok | Frame sent and ACKed |
| k_rx_err_invalid_arg | link, payload NULL or payload_len > 1024 |
| k_rx_err_invalid_state | link not initialized |
| k_rx_err_retry_limit | All retries exhausted (max NACKs/timeouts) |
| k_rx_err_timeout | No ACK received on any attempt |

Precondition

link must be initialized

payload_len <= k_harq_max_payload (1024)

Postcondition

On success: frame delivered and acknowledged

On failure: link state set to error (call rx_spi_link_reset)

Note

Blocking: waits up to max_retries * ack_timeout_ms

Thread Safety: Not thread-safe. Serialize with external mutex.

See also

rx_spi_link_receive() Receive path

Since

Version 1.0.0

See also

rx_spi_link.h for full documentation

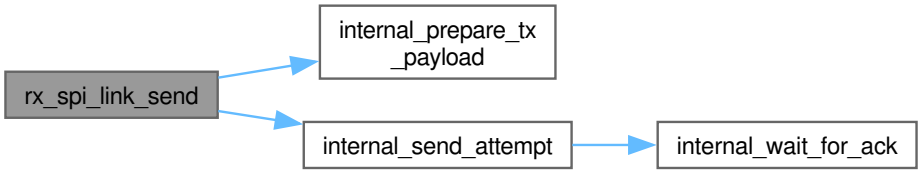
Definition at line 501 of file rx_spi_link.c.

```
00505 {
00506   /* Pre-condition 1: NULL and state checks */
00507   if (link == nullptr) {
00508     return k_rx_err_invalid_arg;
00509   }
00510   if (!link->initialized) {
00511     return k_rx_err_invalid_state;
00512   }
00513
00514   /* Pre-condition 2: Payload validation */
00515   if (payload == nullptr && payload_len > 0) {
00516     return k_rx_err_invalid_arg;
00517   }
00518   if (payload_len > k_harq_max_payload) {
00519     rx_log_error(s_tag, "Payload too large for HARQ");
00520     return k_rx_err_invalid_size;
00521   }
00522
00523   /* Reset HARQ for new transaction. Fails only if harq is null/uninitialized;
00524    * both are invariants checked above via link->initialized. */
00525   (void)rx_harq_reset(&link->harq);
00526
00527   /* Prepare payload: FEC encode if enabled, otherwise passthrough.
00528    * internal_prepare_tx_payload only fails if FEC encode fails, which is
00529    * prevented by the payload_len <= k_harq_max_payload check above. */
00530   const uint8_t* tx_payload = nullptr;
00531   uint32_t       tx_len     = 0;
00532   uint8_t        base_flags = 0;
00533
00534   (void)internal_prepare_tx_payload(link, payload, payload_len, &tx_payload, &tx_len, &base_flags);
00535
00536   /* Retry loop (bounded by max_retries)
00537      * NOTE: This function is NOT thread-safe. Caller must ensure that
00538      * rx_spi_link_send() is not called concurrently on the same link handle.
00539      * Typical enforcement: Single task owns link, or external mutex protection. */
00540   for (uint8_t attempt = 0; attempt < link->max_retries; attempt++) {
00541     const rx_err_t err = internal_send_attempt(link, type, base_flags, tx_payload, tx_len, attempt);
00542
00543     if (err == k_rx_ok) {
00544       /* ACK received - success */
00545       link->state = k_spi_link_state_idle;
00546       return k_rx_ok;
00547     }
00548
00549     if (err == k_rx_err_protocol_error) {
00550       /* NACK received - gateway decode failed, retry */
00551       rx_log_debug(s_tag, "NACK received, retrying");
```

```
00552       continue;
00553     }
00554
00555     /* Timeout or send error - retry */
00556     rx_log_debug(s_tag, "Send attempt failed, retrying");
00557   }
00558
00559   /* All retries exhausted */
00560   link->state = k_spi_link_state_error;
00561   rx_log_error(s_tag, "Max retries exceeded");
00562   return k_rx_err_retry_limit;
00563 }
```

References rx_spi_link_t::harq, rx_spi_link_t::initialized, internal_prepare_tx_payload(), internal_send_attempt(), k_spi_link_state_error, k_spi_link_state_idle, rx_spi_link_t::max_retries, s_tag, and rx_spi_link_t::state.

Here is the call graph for this function:




## 4.2 rx_spi_link.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_spi_link.h
00003  * @brief SPI Link Layer with HARQ
00004  *
00005  * @details
00006  * Implements a reliability layer between the communication manager and the raw
00007  * SPI transport. This module mirrors the Go gateway's `internal/link/spi.go`,
00008  * providing:
00009  *
00010  * - **FEC encoding** on TX (convolutional K=7, rate 1/2)
00011  * - **FEC decoding** on RX (soft-decision Viterbi)
00012  * - **Chase Combining** across retransmissions (soft bit accumulation)
00013  * - **ACK/NACK handshake** with sequence tracking
00014  * - **Configurable retries** (default 3)
00015  *
00016  * ## Architecture
00017  *
00018  * ```
00019  * rx_comm_manager
00020  *       |
00021  *       v
00022  * +--------------+
00023  * | rx_spi_link  |  <-- THIS MODULE (HARQ)
00024  * |              |
00025  * | FEC encode   |  TX: payload -> FEC encode -> frame -> SPI
00026  * | FEC decode   |  RX: SPI -> frame -> soft bits -> Viterbi -> payload
00027  * | Chase Cmb    |  Retry: combine soft bits across attempts
00028  * | ACK/NACK     |  Handshake with gateway
00029  * +--------------+
00030  *       |
00031  *       v
00032  * rx_spi_comm (frame + CRC-32 + raw SPI)
00033  * ```
00034  *
00035  * ## Protocol Flow (TX)
00036  *
00037  * 1. Application calls rx_spi_link_send(payload)
00038  * 2. If FEC enabled: rx_harq_encode(payload) -> encoded_payload (2N+2 bytes)
00039  * 3. rx_spi_comm_send(encoded_payload, flags=FEC_ENABLED|REQUIRES_ACK)
00040  * 4. Wait for ACK/NACK from gateway (via rx_spi_comm_receive)
00041  * 5. On NACK or timeout: retransmit (up to max_retries)
00042  * 6. On ACK: return success
00043  *
00044  * ## Protocol Flow (RX)
00045  *
00046  * 1. rx_spi_comm_receive() returns a frame
```

```
00047  * 2. If ACK/NACK frame: dispatch to pending TX operation
00048  * 3. If data frame with FEC flag: convert payload bytes to soft bits
00049  * 4. Pass soft bits to rx_harq_decode() (Chase Combining + Viterbi)
00050  * 5. If decode succeeds: send ACK, return decoded payload
00051  * 6. If decode fails and retries remain: store in combiner, send NACK
00052  * 7. If max retries exceeded: return error
00053  *
00054  * ## Comparison: USB CDC vs SPI Link
00055  *
00056  * | Feature              | USB CDC (rx_usb_comm) | SPI Link (rx_spi_link)      |
00057  * |----------------------|-----------------------|-----------------------------|
00058  * | FEC encoding         | No                    | Yes (K=7, rate 1/2)         |
00059  * | Viterbi decoding     | No                    | Yes (soft-decision)         |
00060  * | Chase Combining      | No                    | Yes (int16 accumulation)    |
00061  * | ACK/NACK             | No (USB HW handles)   | Yes (explicit handshake)    |
00062  * | Retransmission       | No (USB HW handles)   | Yes (max 3 attempts)        |
00063  * | CRC-32               | Yes                   | Yes (frame layer)           |
00064  * | Sequence tracking    | Yes (shared session)  | Yes (shared session)        |
00065  *
00066  * ## Memory Requirements
00067  *
00068  * | Resource             | Size     | Notes                              |
00069  * |----------------------|----------|------------------------------------|
00070  * | rx_spi_link_t        | ~86 KB   | Dominated by HARQ handle (82 KB)   |
00071  * | FEC encode buffer    | 2050 B   | Max encoded payload (1024*2+2)     |
00072  * | Soft bits buffer     | 16400 B  | Part of HARQ handle                |
00073  * | Stack (send)         | ~256 B   | Function locals + encode buffer    |
00074  * | Stack (receive)      | ~128 B   | Function locals                    |
00075  *
00076  * @par NASA Power of 10 Compliance
00077  * - Rule 1: No goto, setjmp/longjmp, recursion
00078  * - Rule 2: All loops bounded (max_retries, k_fec_max_symbols)
00079  * - Rule 3: Zero dynamic allocation (all static buffers)
00080  * - Rule 4: Functions < 60 lines
00081  * - Rule 5: Minimum 2 pre/postconditions per function
00082  * - Rule 8: C23 typed enums for all constants
00083  * - Rule 9: Function pointers for dependency inversion (INTENTIONAL)
00084  * - Rule 10: Compiled with -Wall -Wextra -Werror
00085  *
00086  * @par SOLID Principles
00087  * - **S**: Only handles HARQ reliability (not framing, not transport)
00088  * - **O**: Configurable via rx_spi_link_config_t without modifying code
00089  * - **L**: Substitutable for rx_spi_comm (same send/receive semantics)
00090  * - **I**: Small focused API (init, deinit, send, receive, reset)
00091  * - **D**: Depends on rx_spi_comm and rx_harq abstractions
00092  *
00093  * @see star-gateway/internal/link/spi.go   Go reference implementation
00094  * @see rx_harq.h                          HARQ protocol and Chase Combiner
00095  * @see rx_fec.h                           FEC encoder/decoder
00096  * @see rx_spi_comm.h                      Raw SPI transport with framing
00097  *
00098  * @author Locked, Inc.
00099  * @date 2026-02-14
00100  * @copyright Copyright (c) 2026 Locked Inc.
00101  * SPDX-License-Identifier: MIT
00102  *
00103  * @since Version 1.0.0
00104  */
00105
00106 #pragma once
00107
00108 #include <stdbool.h>
00109 #include <stdint.h>
00110
00111 #include "rx_err.h"
00112 #include "rx_frame.h"
00113 #include "rx_harq.h"
00114 #include "rx_spi_comm.h"
00115
00116 #ifdef __cplusplus
00117 extern "C" {
00118 #endif
00119
00120 /*
      ================================================================================
00121  * Constants
00122  *
      ================================================================================
      */
00123
00124 /**
00125  * @enum rx_spi_link_constants_t
00126  * @brief SPI link layer compile-time constants
00127  *
00128  * @details
00129  * Protocol timing and retry parameters matching the Go gateway's
00130  * `internal/link/spi.go` constants.
```

```
00131  *
00132  * @invariant max_retries must be in range [0, 255], where 0 means use default value of 3
00133  * @invariant fec_enabled is boolean (0 or 1)
00134  *
00135  * @code
00136  * // Example: Using default constants
00137  * rx_spi_link_config_t config = {
00138  *     .spi_handle = &spi_handle,
00139  *     .fec_enabled = k_spi_link_default_fec_enabled,  // 1 (enabled)
00140  *     .max_retries = k_spi_link_default_max_retries,  // 3 attempts
00141  * };
00142  * @endcode
00143  *
00144  * @see star-gateway/internal/link/spi.go Go reference constants (maxRetries=3)
00145  * @see rx_spi_link_config_t Configuration structure using these defaults
00146  * @since Version 1.0.0
00147  */
00148 typedef enum : uint8_t {
00149   k_spi_link_default_max_retries =
00150     3, /**< Default maximum transmission attempts (matches Go maxRetries=3). After 3 failed attempts (including Chase Combining), the link returns an error and the comm_manager may fail over to USB. */
00151
00152   k_spi_link_default_fec_enabled =
00153     1, /**< Default FEC enabled state (enabled by default, aligns with Go HARQ). FEC/HARQ is enabled by default for reliability. Can be disabled at runtime via config or test modes for raw SPI operation. */
00154 } rx_spi_link_constants_t;
00155
00156 /**
00157  * @enum rx_spi_link_timing_t
00158  * @brief SPI link layer timing constants
00159  *
00160  * @details
00161  * Timeout values for ACK waiting and buffer sizing. Matches Go gateway's
00162  * ackTimeout = 10ms and FEC encoding buffer requirements.
00163  *
00164  * @invariant ack_timeout_ms must be > 0 and < 65535 milliseconds
00165  * @invariant max_encoded_payload = 2 * k_harq_max_payload + k_fec_tail_bits (2050 bytes for 1024-byte max payload)
00166  *
00167  * @code
00168  * // Example: Using timing constants for ACK wait
00169  * rx_err_t err = rx_spi_comm_receive(spi_handle, &ack_frame, k_spi_link_ack_timeout_ms);
00170  * if (err == k_rx_err_timeout) {
00171  *     // No ACK received within 10ms, retry transmission
00172  * }
00173  * @endcode
00174  *
00175  * @see star-gateway/internal/link/spi.go Go ackTimeout constant (10ms)
00176  * @see rx_fec.h FEC encoding parameters (rate 1/2, tail bits)
00177  * @since Version 1.0.0
00178  */
00179 typedef enum : uint16_t {
00180   k_spi_link_ack_timeout_ms =
00181     10, /**< ACK/NACK wait timeout in milliseconds (matches Go ackTimeout=10ms). How long to wait for ACK/NACK after sending a frame. If no response arrives within this window, the frame is retransmitted. */
00182 } rx_spi_link_timing_t;
00183
00184 /**
00185  * @enum rx_spi_link_buffer_t
00186  * @brief Buffer size constants for SPI link layer
00187  *
00188  * @details
00189  * Defines buffer sizes for FEC-encoded payloads. Sized to accommodate the
00190  * maximum payload after FEC encoding (rate 1/2) plus tail bits.
00191  *
00192  * **Formula Derivation:**
00193  * - FEC rate 1/2: Each input byte produces 2 output bytes
00194  * - k_harq_max_payload = 1024 bytes
00195  * - FEC encoded size = 2 * k_harq_max_payload = 2 * 1024 = 2048 bytes
00196  * - k_fec_tail_bits = 6 bits (trellis termination, K=7 constraint length)
00197  * - Tail bytes = ceiling(k_fec_tail_bits / 8) = ceiling(6 / 8) = 1 byte
00198  * - Total = 2048 + 1 = 2049 bytes
00199  *
00200  * @see rx_harq.h HARQ protocol maximum payload size
00201  * @see rx_fec.h FEC encoding parameters (rate 1/2, tail bits)
00202  * @since Version 1.0.0
00203  */
00204 typedef enum : uint16_t {
00205   k_spi_link_max_encoded_payload =
00206     2049, /**< Maximum FEC-encoded payload size in bytes. Formula: bytes = (2 * k_harq_max_payload) + ceiling(k_fec_tail_bits / 8) = (2 * 1024) + ceiling(6 / 8) = 2048 + 1 = 2049. FEC rate 1/2 doubles the payload size, and tail bits (6 bits = 1 byte after ceiling) are appended for trellis termination. */
00207 } rx_spi_link_buffer_t;
00208
00209 /*
    ============================================================================
00210  * State Machine
```

```
00211 *
      ================================================================================
      */
00212
00213 /**
00214 * @enum rx_spi_link_state_t
00215 * @brief SPI link layer state machine states
00216 *
00217 * @details
00218 * Mirrors the Go gateway's harq.State enum. Tracks whether the link is
00219 * idle, waiting for acknowledgment, retransmitting, or in error.
00220 *
00221 * @startuml
00222 * [*] --> Idle
00223 *
00224 * state Idle {
00225 *   Idle : entry / clear_retry_count()
00226 *   Idle : exit / none
00227 * }
00228 *
00229 * state WaitingAck {
00230 *   WaitingAck : entry / start_ack_timer()
00231 *   WaitingAck : exit / stop_ack_timer()
00232 * }
00233 *
00234 * state Retransmitting {
00235 *   Retransmitting : entry / increment_retry_count()
00236 *   Retransmitting : exit / log_retry_attempt()
00237 * }
00238 *
00239 * state Error {
00240 *   Error : entry / set_error_flag(), log_failure()
00241 *   Error : exit / clear_error_flag()
00242 * }
00243 *
00244 * Idle --> WaitingAck : send() / encode_payload()
00245 * WaitingAck --> Idle : ACK received / clear_retry_count()
00246 * WaitingAck --> Retransmitting : NACK or timeout / [retry_count < max]
00247 * Retransmitting --> WaitingAck : retransmit sent / update_sequence()
00248 * Retransmitting --> Error : [retry_count >= max_retries]
00249 * Error --> Idle : reset() / clear_state()
00250 * @enduml
00251 *
00252 * @par State Transition Table
00253 *
00254 * | From State      | To State        | Event/Condition          | Guard/Action                | Entry Action             | Exit
      Action         |
00255 * |----------------|----------------|--------------------------|-----------------------------|--------------------------|--------------------|
00256 * | Idle           | WaitingAck      | send() called            | encode_payload()            | start_ack_timer()        | none
      |
00257 * | WaitingAck     | Idle            | ACK received             | clear_retry_count()         | clear_retry_count()      |
      stop_ack_timer()    |
00258 * | WaitingAck     | Retransmitting  | NACK or timeout          | retry_count < max_retries   |
      increment_retry_count()   | stop_ack_timer()    |
00259 * | Retransmitting | WaitingAck      | retransmit sent          | update_sequence()           | start_ack_timer()        |
      log_retry_attempt() |
00260 * | Retransmitting | Error           | max retries exceeded     | retry_count >= max_retries  | set_error_flag(), log()  |
      log_retry_attempt() |
00261 * | Error          | Idle            | reset() called           | clear_state()               | clear_retry_count()      |
      clear_error_flag()  |
00262 *
00263 * @see rx_spi_link_get_state() Query current state
00264 * @see rx_spi_link_reset() Reset to Idle state
00265 * @since Version 1.0.0
00266 */
00267 typedef enum : uint8_t {
00268   k_spi_link_state_idle          = 0, /**< Ready for new send/receive */
00269   k_spi_link_state_waiting_ack   = 1, /**< Frame sent, waiting for ACK/NACK */
00270   k_spi_link_state_retransmitting = 2, /**< Retransmitting after NACK/timeout */
00271   k_spi_link_state_error         = 3, /**< Unrecoverable error (max retries) */
00272 } rx_spi_link_state_t;
00273
00274 /*
      ================================================================================
00275 * Types
00276 *
      ================================================================================
      */
00277
00278 /**
00279 * @struct rx_spi_link_config_t
00280 * @brief SPI link layer configuration
00281 *
00282 * @details
00283 * Passed to rx_spi_link_init() to configure the link behavior. All zero
00284 * values use defaults.
00285 *
```

```
00286  * @invariant spi_handle must be non-NULL and initialized via rx_spi_comm_init()
00287  * @invariant max_retries in range [0, 255], where 0 means use default (3)
00288  * @invariant fec_enabled is boolean (0 or 1)
00289  *
00290  * @par Example
00291  * @code{.c}
00292  * rx_spi_link_config_t cfg = {
00293  *     .spi_handle  = &g_spi_comm,
00294  *     .fec_enabled = true,
00295  *     .max_retries = 3,
00296  * };
00297  * @endcode
00298  *
00299  * @see rx_spi_link_init() Initialization function consuming this config
00300  * @see rx_spi_comm_handle_t Underlying SPI transport handle type
00301  * @since Version 1.0.0
00302  */
00303 typedef struct {
00304   rx_spi_comm_handle_t*
00305     spi_handle; /**< Underlying SPI transport (REQUIRED, must be non-NULL and initialized via rx_spi_comm_init) */
00306   bool
00307     fec_enabled; /**< Enable FEC encoding/decoding (true=HARQ with Chase Combining, false=raw frames) */
00308   uint8_t max_retries; /**< Maximum transmission attempts (0=use default of 3, range [0-255]) */
00309 } rx_spi_link_config_t;
00310
00311 /**
00312  * @struct rx_spi_link_receive_result_t
00313  * @brief Result of a successful receive operation
00314  *
00315  * @details
00316  * Contains the decoded payload and metadata about the decoding process.
00317  * Mirrors Go's harq.ReceiveResult.
00318  *
00319  * @warning This structure contains a large 1024-byte payload buffer (total struct
00320  * size ~1032 bytes). MUST be statically allocated (global or static variable) only.
00321  * DO NOT allocate on the stack (causes stack overflow when passing to rx_spi_link_receive()).
00322  * DO NOT use dynamic allocation (malloc/new) - zero heap usage required per NASA Rule 3 and
00323  * project coding guidelines. All buffers must use enum-defined sizes for compile-time allocation.
00324  *
00325  * @invariant payload_len <= k_harq_max_payload (1024 bytes)
00326  * @invariant sequence in range [0, 65535] (wraps around)
00327  * @invariant combining_count in range [0, max_retries], typically [1-4]
00328  *
00329  * @see rx_spi_link_receive() Function that populates this result structure
00330  * @see star-gateway/internal/harq/harq.go Go equivalent (ReceiveResult)
00331  * @since Version 1.0.0
00332  */
00333 typedef struct {
00334   uint8_t payload
00335     [k_harq_max_payload]; /**< Decoded payload data (buffer size: k_harq_max_payload = 1024 bytes) */
00336   uint32_t payload_len;   /**< Decoded payload length in bytes (range: [0, k_harq_max_payload]) */
00337   uint16_t sequence;      /**< Frame sequence number (range: [0, 65535], wraps around) */
00338   rx_frame_type_t
00339       frame_type;  /**< Frame type (e.g., k_frame_type_command, k_frame_type_telemetry) */
00340   bool fec_decoded; /**< True if FEC/Viterbi decoding was applied, false if raw passthrough */
00341   uint8_t
00342     combining_count; /**< Number of Chase Combining attempts used (1 = first attempt, >1 = retransmissions combined)
      */
00343   bool
00344     is_retransmit; /**< True if frame had k_frame_flag_retransmit set, false for first transmission */
00345 } rx_spi_link_receive_result_t;
00346
00347 /**
00348  * @struct rx_spi_link_t
00349  * @brief SPI link layer handle
00350  *
00351  * @details
00352  * Contains the HARQ state machine, FEC encoder/decoder, Chase Combiner,
00353  * and a pointer to the underlying SPI transport. Approximately 86 KB
00354  * total (dominated by the HARQ handle's FEC decoder buffers).
00355  *
00356  * ## Memory Layout
00357  *
00358  * | Field       | Size    | Description                          |
00359  * |-------------|---------|--------------------------------------|
00360  * | spi_handle  | 8 B     | Pointer to SPI transport             |
00361  * | harq        | ~82 KB  | HARQ handle (FEC + Chase Combiner)   |
00362  * | state       | 1 B     | Current link state                   |
00363  * | fec_enabled | 1 B     | FEC enable flag                      |
00364  * | max_retries | 1 B     | Max transmission attempts            |
00365  * | initialized | 1 B     | Initialization flag                  |
00366  * | fec_buf     | 2050 B  | FEC encode output buffer             |
00367  *
00368  * @invariant When initialized == true, spi_handle must be non-NULL
00369  * @invariant state must be one of k_spi_link_state_* values
00370  * @invariant max_retries in range [1, 255] when initialized (0 converted to 3 at init)
00371  * @invariant fec_encode_buf size == k_spi_link_max_encoded_payload (2050 bytes)
```

```
00372  *
00373  * @warning This structure is ~86 KB. Allocate statically, not on the stack.
00374  *
00375  * @see rx_spi_link_init() Initialization function
00376  * @see rx_spi_link_config_t Configuration structure
00377  * @see rx_harq_handle_t HARQ handle containing FEC encoder/decoder
00378  * @since Version 1.0.0
00379  */
00380 typedef struct {
00381   rx_spi_comm_handle_t*
00382     spi_handle; /**< Underlying SPI transport (must be non-NULL when initialized, set via config) */
00383   rx_harq_handle_t
00384     harq; /**< HARQ handle containing FEC encoder/decoder and Chase Combiner (~82 KB) */
00385   rx_spi_link_state_t state; /**< Current link state (idle, waiting_ack, retransmitting, error) */
00386   bool fec_enabled; /**< FEC enabled flag (true=HARQ, false=raw frames, immutable after init) */
00387   uint8_t
00388     max_retries; /**< Maximum transmission attempts (range [1-255], 0 at init converted to 3) */
00389   bool
00390     initialized; /**< Initialization flag (true=ready for use, false=must call rx_spi_link_init) */
00391
00392   uint8_t fec_encode_buf
00393     [k_spi_link_max_encoded_payload]; /**< Staging buffer for FEC-encoded payloads (size: 2049 bytes = 2*1024+1).
       Used by rx_spi_link_send() to hold FEC-encoded data before passing to rx_spi_comm_send(). Formula: 2 *
       k_harq_max_payload + ceiling(k_fec_tail_bits / 8), where k_fec_tail_bits = 6. */
00394 } rx_spi_link_t;
00395
00396 /*
       ============================================================================
00397  * Lifecycle API
00398  *
       ============================================================================
       */
00399
00400 /**
00401  * @brief Initialize SPI link layer
00402  *
00403  * @details
00404  * Initializes the HARQ handle (including FEC encoder/decoder and Chase
00405  * Combiner) and stores configuration. The underlying SPI transport must
00406  * already be initialized.
00407  *
00408  * @param[out] link   Link handle to initialize
00409  * @param[in]  config Configuration (spi_handle is REQUIRED)
00410  *
00411  * @return rx_err_t Error code
00412  * @retval k_rx_ok Success
00413  * @retval k_rx_err_invalid_arg link or config isnullptr; config->spi_handle isnullptr
00414  *
00415  * @pre link must point to valid, allocated memory (~86 KB)
00416  * @pre config->spi_handle must be initialized via rx_spi_comm_init()
00417  * @post link->initialized == true
00418  * @post link->harq initialized with FEC if fec_enabled
00419  *
00420  * @note Thread Safety: Not thread-safe. Call from single thread during init.
00421  *
00422  * @see rx_spi_link_deinit() Cleanup counterpart
00423  * @since Version 1.0.0
00424  */
00425 [[nodiscard]] rx_err_t rx_spi_link_init(rx_spi_link_t* link, const rx_spi_link_config_t* config);
00426
00427 /**
00428  * @brief Deinitialize SPI link layer
00429  *
00430  * @details
00431  * Deinitializes the HARQ handle and marks the link as uninitialized.
00432  * Does NOT deinitialize the underlying SPI transport.
00433  *
00434  * @param[in,out] link Link handle to deinitialize
00435  *
00436  * @return rx_err_t Error code
00437  * @retval k_rx_ok Success
00438  * @retval k_rx_err_invalid_arg link isnullptr
00439  * @retval k_rx_err_invalid_state link not initialized
00440  *
00441  * @pre link must be non-NULL
00442  * @pre link must be initialized (link->initialized == true)
00443  * @post link->initialized == false
00444  * @post link->harq internal state deinitialized (HARQ handle invalid)
00445  *
00446  * @note Does NOT deinitialize the underlying SPI transport handle
00447  *
00448  * @see rx_spi_link_init() Initialization counterpart
00449  * @see rx_harq_deinit() Internal HARQ deinitialization
00450  * @since Version 1.0.0
00451  */
00452 [[nodiscard]] rx_err_t rx_spi_link_deinit(rx_spi_link_t* link);
00453
```

```
00454 /*
      ============================================================================
00455  * Send API
00456  *
      ============================================================================
      */
00457
00458 /**
00459  * @brief Send payload with HARQ reliability over SPI
00460  *
00461  * @details
00462  * Encodes payload with FEC (if enabled), sends via SPI transport, and
00463  * waits for ACK/NACK. On NACK or timeout, retransmits up to max_retries.
00464  *
00465  * ## Algorithm
00466  *
00467  * 1. Validate parameters
00468  * 2. If FEC enabled: rx_harq_encode(payload) -> encoded (2N+2 bytes)
00469  * 3. For attempt = 1 to max_retries:
00470  *    a. Build frame with REQUIRES_ACK | FEC_ENABLED flags
00471  *    b. If retransmit: add RETRANSMIT flag
00472  *    c. Send via rx_spi_comm_send()
00473  *    d. Wait for ACK/NACK (rx_spi_comm_receive with ack_timeout_ms)
00474  *    e. On ACK: return success
00475  *    f. On NACK/timeout: continue to next attempt
00476  * 4. All retries exhausted: return error
00477  *
00478  * @param[in,out] link        Initialized link handle
00479  * @param[in]     type        Frame type (k_frame_type_command, etc.)
00480  * @param[in]     payload     Data to send
00481  * @param[in]     payload_len Data length (0-1024 bytes)
00482  *
00483  * @return rx_err_t Error code
00484  * @retval k_rx_ok Frame sent and ACKed
00485  * @retval k_rx_err_invalid_arg link, payload NULL or payload_len > 1024
00486  * @retval k_rx_err_invalid_state link not initialized
00487  * @retval k_rx_err_retry_limit All retries exhausted (max NACKs/timeouts)
00488  * @retval k_rx_err_timeout No ACK received on any attempt
00489  *
00490  * @pre link must be initialized
00491  * @pre payload_len <= k_harq_max_payload (1024)
00492  * @post On success: frame delivered and acknowledged
00493  * @post On failure: link state set to error (call rx_spi_link_reset)
00494  *
00495  * @note Blocking: waits up to max_retries * ack_timeout_ms
00496  * @note Thread Safety: Not thread-safe. Serialize with external mutex.
00497  *
00498  * @see rx_spi_link_receive() Receive path
00499  * @since Version 1.0.0
00500  */
00501 [[nodiscard]] rx_err_t rx_spi_link_send(rx_spi_link_t*  link,
00502                                         rx_frame_type_t type,
00503                                         const uint8_t*  payload,
00504                                         uint32_t        payload_len);
00505
00506 /*
      ============================================================================
00507  * Receive API
00508  *
      ============================================================================
      */
00509
00510 /**
00511  * @brief Receive and decode a frame from SPI with HARQ
00512  *
00513  * @details
00514  * Receives a frame from the SPI transport, handles ACK/NACK dispatch,
00515  * and performs FEC decoding with Chase Combining if applicable.
00516  *
00517  * ## Algorithm
00518  *
00519  * 1. Call rx_spi_comm_receive() with timeout
00520  * 2. If frame is ACK/NACK: handle internally (dispatch), return no-data
00521  * 3. If frame is PING: auto-respond with PONG (handled by spi_comm)
00522  * 4. If data frame with FEC flag:
00523  *    a. Convert payload bytes to soft bits (byte -> 8 soft bits)
00524  *    b. If retransmit flag: add to Chase Combiner, decode combined
00525  *    c. If first attempt: decode directly, store in combiner on failure
00526  *    d. On decode success: send ACK, populate result
00527  *    e. On decode failure: send NACK, return error
00528  * 5. If data frame without FEC: pass payload through directly, send ACK
00529  *
00530  * @param[in,out] link       Initialized link handle
00531  * @param[out]    result     Decoded payload and metadata
00532  * @param[in]     timeout_ms Receive timeout (0 = non-blocking)
00533  *
00534  * @return rx_err_t Error code
```

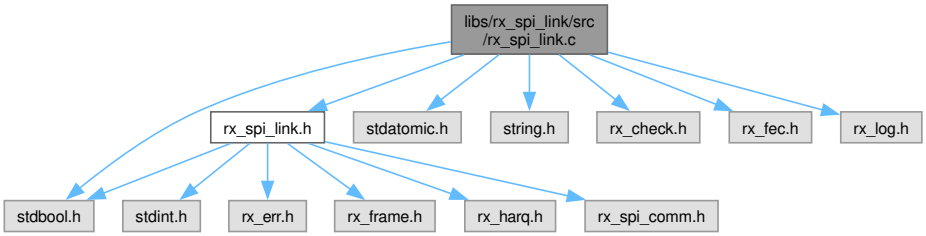
```
00535  * @retval k_rx_ok Frame received and decoded successfully
00536  * @retval k_rx_err_invalid_arg link or result isnullptr
00537  * @retval k_rx_err_invalid_state link not initialized
00538  * @retval k_rx_err_timeout No frame received within timeout
00539  * @retval k_rx_err_protocol_error FEC decode failed (NACK sent)
00540  * @retval k_rx_err_crc_mismatch CRC validation failed at frame layer
00541  *
00542  * @pre link must be initialized
00543  * @pre result must point to valid memory
00544  * @post On success: result populated with decoded payload and metadata
00545  * @post On ACK/NACK frame: returns k_rx_err_timeout (control frame, not data)
00546  *
00547  * @note Thread Safety: Not thread-safe. Dedicate one thread to receiving.
00548  *
00549  * @see rx_spi_link_send() Send path
00550  * @since Version 1.0.0
00551  */
00552 [[nodiscard]] rx_err_t
00553 rx_spi_link_receive(rx_spi_link_t* link, rx_spi_link_receive_result_t* result, uint32_t timeout_ms);
00554
00555 /*
      ============================================================================
00556  * State Management
00557  *
      ============================================================================
      */
00558
00559 /**
00560  * @brief Reset SPI link layer for new session
00561  *
00562  * @details
00563  * Resets the HARQ state machine, clears the Chase Combiner, and returns
00564  * the link to idle state. Does NOT reset the shared session state
00565  * (sequence numbers are managed by rx_session).
00566  *
00567  * @param[in,out] link Initialized link handle
00568  *
00569  * @return rx_err_t Error code
00570  * @retval k_rx_ok Success
00571  * @retval k_rx_err_invalid_arg link isnullptr
00572  * @retval k_rx_err_invalid_state link not initialized
00573  *
00574  * @pre link must be non-NULL
00575  * @pre link must be initialized (link->initialized == true)
00576  * @post link->state == k_spi_link_state_idle
00577  * @post Chase Combiner buffers cleared (all soft-decision history discarded)
00578  *
00579  * @note Does NOT reset TX/RX sequence numbers (managed externally by rx_session)
00580  * @note Safe to call after error state to recover link operation
00581  *
00582  * @see rx_harq_reset() Internal HARQ reset
00583  * @see rx_spi_link_get_state() Query current state
00584  * @since Version 1.0.0
00585  */
00586 [[nodiscard]] rx_err_t rx_spi_link_reset(rx_spi_link_t* link);
00587
00588 /**
00589  * @brief Get current SPI link state
00590  *
00591  * @details
00592  * Returns the current state of the SPI link state machine. Safe to call
00593  * with nullptr (returns error state). Does not modify link state.
00594  *
00595  * @param[in] link Link handle (may be NULL)
00596  *
00597  * @return rx_spi_link_state_t Current link state
00598  * @retval k_spi_link_state_idle Link is ready for new send/receive
00599  * @retval k_spi_link_state_waiting_ack Waiting for ACK/NACK response
00600  * @retval k_spi_link_state_retransmitting Retransmitting after NACK/timeout
00601  * @retval k_spi_link_state_error Unrecoverable error (also returned if link is NULL)
00602  *
00603  * @pre link may be NULL (function handles gracefully)
00604  * @pre If link is non-NULL, may be in any state (initialized or not)
00605  * @post No state change (pure query function)
00606  * @post Return value reflects state at time of call (may change immediately after)
00607  *
00608  * @note Thread-safe: Read-only operation, no synchronization needed
00609  * @note NULL-safe: Returns k_spi_link_state_error when link isnullptr
00610  * @note Does not distinguish between "link is NULL" vs "link is in error state"
00611  *
00612  * @see rx_spi_link_t Link handle structure
00613  * @see rx_spi_link_state_t State enumeration with all possible values
00614  * @see rx_spi_link_reset() Reset link to idle state
00615  *
00616  * @since Version 1.0.0
00617  */
00618 rx_spi_link_state_t rx_spi_link_get_state(const rx_spi_link_t* link);
```

```
00619
00620 /**
00621  * @brief Check if FEC is enabled on this link
00622  *
00623  * @details
00624  * Queries the FEC/HARQ enabled flag. Returns false if link is NULL.
00625  * Does not modify link state. FEC setting is configured at init time
00626  * via rx_spi_link_config_t.fec_enabled and cannot be changed at runtime.
00627  *
00628  * @param[in] link Link handle (may be NULL)
00629  *
00630  * @return bool FEC enabled status
00631  * @retval true FEC encoding/decoding is active (Chase Combining, Viterbi)
00632  * @retval false FEC disabled (raw payload transmission) or link isnullptr
00633  *
00634  * @pre link may be NULL (function handles gracefully by returning false)
00635  * @pre If link is non-NULL, may be in any state (initialized or not)
00636  * @post No state change (pure query function)
00637  * @post Return value reflects FEC setting at time of call (constant for link lifetime)
00638  *
00639  * @note Thread-safe: Read-only operation, no synchronization needed
00640  * @note NULL-safe: Returns false when link isnullptr
00641  * @note FEC setting is immutable after initialization (cannot be changed at runtime)
00642  * @note When FEC is enabled, rx_spi_link_send/receive use HARQ with Chase Combining
00643  *
00644  * @see rx_spi_link_config_t Configuration structure with fec_enabled field
00645  * @see rx_spi_link_init() Initialization function that sets FEC mode
00646  * @see k_spi_link_default_fec_enabled Default FEC setting (enabled = 1)
00647  *
00648  * @since Version 1.0.0
00649  */
00650 bool rx_spi_link_fec_enabled(const rx_spi_link_t* link);
00651
00652 #ifdef ___cplusplus
00653 }
00654 #endif
```

## 4.3 libs/rx_spi_link/src/rx_spi_link.c File Reference

SPI Link Layer with HARQ Implementation.

#include ”rx_spi_link.h”
#include <stdatomic.h>
#include <stdbool.h>
#include <string.h>
#include ”rx_check.h”
#include ”rx_fec.h”
#include ”rx_log.h”
Include dependency graph for rx_spi_link.c:




Enumerations

- enum internal_soft_bit_values_t : int8_t { k_internal_soft_bit_one = 127 , k_internal_soft_bit_zero = -127 }

  Soft bit conversion constants.
- enum internal_bit_constants_t : uint8_t { k_spi_link_bits_per_byte = 8 , k_spi_link_msb_shift = 7 , k_spi_link_bit_mask = 0x01 }

  Bit manipulation constants for byte-to-soft-bit conversion.

Functions

- RX_STATIC_TESTABLE rx_err_t internal_bytes_to_soft_bits (const uint8_t *data, uint32_t data_len, rx_soft_bit_t *soft, uint32_t soft_size, uint32_t *soft_len)

  Convert payload bytes to soft bits for Viterbi decoder.
- RX_STATIC_TESTABLE rx_err_t internal_wait_for_ack (rx_spi_link_t *link, uint16_t expected_seq)

  Wait for ACK/NACK response from gateway.
- rx_err_t rx_spi_link_init (rx_spi_link_t *link, const rx_spi_link_config_t *config)

  Initialize SPI link layer.
- rx_err_t rx_spi_link_deinit (rx_spi_link_t *link)

  Deinitialize SPI link layer.
- RX_STATIC_TESTABLE rx_err_t internal_send_attempt (rx_spi_link_t *link, rx_frame_type_t type, uint8_t base_flags, const uint8_t *tx_payload, uint32_t tx_len, uint8_t attempt)

  Attempt a single transmission with ACK/NACK handling.
- RX_STATIC_TESTABLE rx_err_t internal_prepare_tx_payload (rx_spi_link_t *link, const uint8_t *payload, uint32_t payload_len, const uint8_t **out_payload, uint32_t *out_len, uint8_t *out_flags)

  Prepare payload for transmission (FEC encode or passthrough).
- rx_err_t rx_spi_link_send (rx_spi_link_t *link, rx_frame_type_t type, const uint8_t *payload, uint32_t payload_len)

  Send payload with HARQ reliability over SPI.
- static uint32_t internal_fec_decoded_len (uint32_t soft_len)

  Calculate expected decoded payload length from FEC soft bit count.
- RX_STATIC_TESTABLE rx_err_t internal_receive_fec_decode (rx_spi_link_t *link, const rx_frame_t *frame, rx_spi_link_receive_result_t *result)

  Process received frame with FEC decoding and Chase Combining.
- RX_STATIC_TESTABLE rx_err_t internal_receive_passthrough (rx_spi_link_t *link, const rx_frame_t *frame, rx_spi_link_receive_result_t *result)

  Process received frame without FEC (passthrough mode).
- rx_err_t rx_spi_link_receive (rx_spi_link_t *link, rx_spi_link_receive_result_t *result, uint32_t timeout_ms)

  Receive and decode a frame from SPI with HARQ.
- rx_err_t rx_spi_link_reset (rx_spi_link_t *link)

  Reset SPI link layer for new session.
- rx_spi_link_state_t rx_spi_link_get_state (const rx_spi_link_t *link)

  Get current SPI link state.
- bool rx_spi_link_fec_enabled (const rx_spi_link_t *link)

  Check if FEC is enabled on this link.

Variables

- RX_STATIC_TESTABLE atomic_flag s_send_path_busy = ATOMIC_FLAG_INIT
- static const char s_tag [] = "SPI_LINK"

  Log tag for SPI link layer messages.
- static const rx_spi_link_t s_zero_link = {}

  Zero-initialized SPI link template for stack-safe initialization.

### 4.3.1 Detailed Description

SPI Link Layer with HARQ Implementation.

Implements the SPI link layer that sits between the communication manager and the raw SPI transport (rx_spi_comm). This is a C port of the Go gateway's internal/link/spi.go, providing:

- FEC encoding on TX (convolutional K=7, rate 1/2)
- Soft-decision Viterbi decoding on RX
- Chase Combining across retransmissions
- ACK/NACK handshake with configurable retries

#### 4.3.1.1 Key Design Decisions

1. Hard-to-soft conversion: SPI is digital (no analog front-end), so received bytes are converted to hard soft bits: 1 -> +127, 0 -> -127. Chase Combining still provides gain because noise corrupts different bits on each retransmission.
2. Blocking send: rx_spi_link_send() blocks until ACK or all retries are exhausted. The comm_task runs at 100 Hz, giving ~10 ms per cycle. With k_spi_link_ack_timeout_ms = 10 ms and 3 retries, worst case is ~30 ms (occupies 3 comm cycles).
3. Non-blocking receive: rx_spi_link_receive() defers to rx_spi_comm_receive() with the caller's timeout. Control frames (ACK/NACK) are handled internally and not returned to the caller.

#### 4.3.1.2 Protocol Flow (TX)

```
payload
  |
  +--> rx_harq_encode() --> encoded_payload (2N+2 bytes if FEC)
  |                    |
  +--> (passthrough if no FEC)
  |
  v
for attempt = 1..3:
  rx_spi_comm_send(encoded_payload, REQUIRES_ACK | FEC_ENABLED)
  wait for ACK/NACK (10 ms timeout)
  ACK -> return success
  NACK/timeout -> set RETRANSMIT flag, retry
```

#### 4.3.1.3 Protocol Flow (RX)

```
rx_spi_comm_receive()
  |
  +--> ACK/NACK -> dispatch internally, return "no data"
  |
  +--> data frame (FEC flag set):
  |      convert bytes to soft bits
  |      rx_harq_decode() (with Chase Combining)
  |      success -> send ACK, return decoded payload
  |      failure -> send NACK, return error
  |
  +--> data frame (no FEC flag):
         passthrough, send ACK, return payload
```

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp/longjmp, recursion
- Rule 2: [OK] All loops bounded (max_retries, k_rx_bits_per_byte)
- Rule 3: [OK] Zero dynamic allocation
- Rule 4: [OK] All functions < 60 lines
- Rule 5: [OK] Minimum 2 preconditions per function
- Rule 6: [OK] Variables at smallest scope
- Rule 7: [OK] All return values checked
- Rule 8: [OK] C23 typed enums for all constants
- Rule 10: [OK] Compiled with -Wall -Wextra -Werror

See also

rx_spi_link.h Public API

star-gateway/internal/link/spi.go Go reference implementation

Author

Locked, Inc.

Date

2026-02-14

Copyright



Since

Version 1.0.0

Definition in file rx_spi_link.c.

### 4.3.2 Enumeration Type Documentation

#### 4.3.2.1 internal_bit_constants_t

enum internal_bit_constants_t : uint8_t

Bit manipulation constants for byte-to-soft-bit conversion.

Since

Version 1.0.0

Enumerator

| | |
|---|---|
| k_spi_link_bits_per_byte | Number of bits in a byte |
| k_spi_link_msb_shift | Shift to extract MSB |
| k_spi_link_bit_mask | Mask to extract single bit |

Definition at line 154 of file rx_spi_link.c.

```
00154             : uint8_t {
00155     k_spi_link_bits_per_byte = 8,    /**< Number of bits in a byte */
00156     k_spi_link_msb_shift     = 7,    /**< Shift to extract MSB */
00157     k_spi_link_bit_mask      = 0x01, /**< Mask to extract single bit */
00158 } internal_bit_constants_t;
```

#### 4.3.2.2 internal_soft_bit_values_t

enum internal_soft_bit_values_t : int8_t

Soft bit conversion constants.

Used when converting received hard bytes to soft bits for the Viterbi decoder. Each bit becomes +127 (confident 1) or -127 (confident 0). Matches Go's bytesToSoftBits() in internal/link/spi.go.

Since

Version 1.0.0

Enumerator

| | |
|---|---|
| k_internal_soft_bit_one | Hard bit 1 -> soft confidence +127 |
| k_internal_soft_bit_zero | Hard bit 0 -> soft confidence -127 |

Definition at line 143 of file rx_spi_link.c.

```
00143             : int8_t {
00144     k_internal_soft_bit_one  = 127,  /**< Hard bit 1 -> soft confidence +127 */
00145     k_internal_soft_bit_zero = -127, /**< Hard bit 0 -> soft confidence -127 */
00146 } internal_soft_bit_values_t;
```

### 4.3.3 Function Documentation

#### 4.3.3.1 internal_bytes_to_soft_bits()

```
RX_STATIC_TESTABLE rx_err_t internal_bytes_to_soft_bits (
                const uint8_t * data,
                uint32_t data_len,
                rx_soft_bit_t * soft,
                uint32_t soft_size,
                uint32_t * soft_len)
```

Convert payload bytes to soft bits for Viterbi decoder.

Each byte is expanded to 8 soft bits, MSB-first. Bit value 1 becomes +127, bit value 0 becomes -127. This matches the Go gateway's bytesToSoftBits() function.

Precondition

data and soft must be non-NULL

soft_size >= data_len * k_spi_link_bits_per_byte

Postcondition

soft_len == data_len * k_spi_link_bits_per_byte

Since

Version 1.0.0

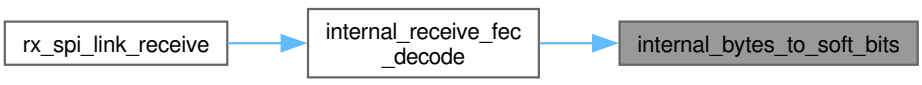
Definition at line 180 of file rx_spi_link.c.

```
00185 {
00186   /* Rule 5: Precondition validation - nullptr checks */
00187   if (data == nullptr) {
00188     if (soft_len != nullptr) {
00189       *soft_len = 0;
00190     }
00191     return k_rx_err_invalid_arg;
00192   }
00193
00194   if (soft == nullptr) {
00195     if (soft_len != nullptr) {
00196       *soft_len = 0;
00197     }
00198     return k_rx_err_invalid_arg;
00199   }
00200
00201   if (soft_len == nullptr) {
00202     return k_rx_err_invalid_arg;
00203   }
00204
00205   /* Rule 5: Precondition validation - overflow check before multiplication.
00206    * data_len > UINT32_MAX/8 means > 536 MB; this is an invariant the caller
00207    * must never violate (protocol enforces max payload of 1024 bytes). */
00208
00209   /* Rule 5: Precondition validation - buffer size check */
00210   const uint32_t required = data_len * (uint32_t)k_spi_link_bits_per_byte;
00211   if (required > soft_size) {
00212     *soft_len = 0;
00213     return k_rx_err_invalid_size;
00214   }
00215
00216   for (uint32_t byte_idx = 0; byte_idx < data_len; byte_idx++) {
00217     const uint8_t b = data[byte_idx];
00218     for (uint8_t bit_idx = 0; bit_idx < k_spi_link_bits_per_byte; bit_idx++) {
00219       const uint8_t  bit = (b » (k_spi_link_msb_shift - bit_idx)) & k_spi_link_bit_mask;
00220       const uint32_t idx = byte_idx * (uint32_t)k_spi_link_bits_per_byte + bit_idx;
00221       soft[idx]          = (bit == k_spi_link_bit_mask) ? (rx_soft_bit_t)k_internal_soft_bit_one
00222                                                         : (rx_soft_bit_t)k_internal_soft_bit_zero;
00223     }
00224   }
00225
00226   *soft_len = required;
00227   return k_rx_ok;
00228 }
```

References k_internal_soft_bit_one, k_internal_soft_bit_zero, k_spi_link_bit_mask, k_spi_link_bits_per_byte, and k_spi_link_msb_shift.

Referenced by internal_receive_fec_decode().

Here is the caller graph for this function:



#### 4.3.3.2 internal_fec_decoded_len()

```
uint32_t internal_fec_decoded_len (
                uint32_t soft_len)    [inline], [static]
```

Calculate expected decoded payload length from FEC soft bit count.

The FEC encoder packs output bits into bytes, rounding up to a whole byte. For N input bytes the encoder produces: total_output_bits = (N*8 + k_fec_tail_bits) * k_fec_num_outputs encoded_bytes = ceil(total_output_bits / 8)

After internal_bytes_to_soft_bits(): soft_len = encoded_bytes * 8 (may include up to 7 padding bits)

Working backwards: num_symbols (including tail) = soft_len / k_fec_num_outputs input_bits (without tail) = num_symbols - k_fec_tail_bits expected_decoded_len = input_bits / k_spi_link_bits_per_byte

Note: integer division truncates, which is correct because the padding bits in the last encoded byte are zero and do not contribute to symbols.

Precondition

soft_len is the output of internal_bytes_to_soft_bits()

soft_len >= k_fec_num_outputs * k_fec_tail_bits (else underflow)

Postcondition

Return value represents the original pre-FEC payload size

Since

Version 1.0.0

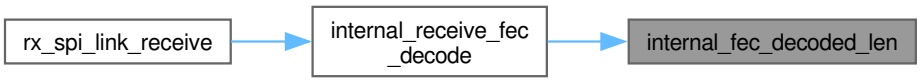
Definition at line 597 of file rx_spi_link.c.

```
00598 {
00599   return (soft_len / (uint32_t)k_fec_num_outputs - (uint32_t)k_fec_tail_bits) /
00600          (uint32_t)k_spi_link_bits_per_byte;
00601 }
```

References k_spi_link_bits_per_byte.

Referenced by internal_receive_fec_decode().

Here is the caller graph for this function:



#### 4.3.3.3 internal_prepare_tx_payload()

```
RX_STATIC_TESTABLE rx_err_t internal_prepare_tx_payload (
                rx_spi_link_t * link,
                const uint8_t * payload,
                uint32_t payload_len,
                const uint8_t ** out_payload,
                uint32_t * out_len,
                uint8_t * out_flags)
```

Prepare payload for transmission (FEC encode or passthrough).

If FEC is enabled and payload is non-empty, encodes via rx_harq_encode() and sets FEC flag. Otherwise, passes payload through unchanged. Sets base_flags to include k_frame_flag_requires_ack and optionally k_frame_flag_fec_enabled.

Precondition

link->fec_enabled indicates whether FEC should be used

link->fec_encode_buf sized for k_spi_link_max_encoded_payload

Postcondition

If FEC enabled: out_payload points to link->fec_encode_buf, out_flags has FEC flag

If FEC disabled: out_payload == payload, out_len == payload_len

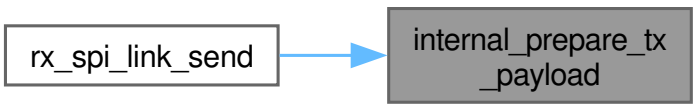
Definition at line 467 of file rx_spi_link.c.

```
00473 {
00474   *out_payload = payload;
00475   *out_len     = payload_len;
00476   *out_flags   = k_frame_flag_requires_ack;
00477
00478   if ((int)link->fec_enabled && payload != nullptr && payload_len > 0) {
00479     uint32_t encoded_len = 0;
00480     (void)(rx_harq_encode(&link->harq,
00481                           payload,
00482                           payload_len,
00483                           link->fec_encode_buf,
00484                           (uint32_t)k_spi_link_max_encoded_payload,
00485                           &encoded_len));
00486     /* rx_harq_encode only fails if payload > k_harq_max_payload or harq
00487      * uninitialized -- both are invariants checked before this call. */
00488
00489     *out_payload = link->fec_encode_buf;
00490     *out_len     = encoded_len;
00491     *out_flags |= k_frame_flag_fec_enabled;
00492   }
00493
00494   return k_rx_ok;
00495 }
```

References rx_spi_link_t::fec_enabled, rx_spi_link_t::fec_encode_buf, rx_spi_link_t::harq, and k_spi_link_max_encoded_payload.

Referenced by rx_spi_link_send().

Here is the caller graph for this function:



#### 4.3.3.4 internal_receive_fec_decode()

```
RX_STATIC_TESTABLE rx_err_t internal_receive_fec_decode (
        rx_spi_link_t * link,
        const rx_frame_t * frame,
        rx_spi_link_receive_result_t * result)
```

Process received frame with FEC decoding and Chase Combining.

Converts hard bits to soft bits, performs Viterbi decoding via HARQ, and sends ACK/NACK based on decode success. On success, resets HARQ combiner. On failure, sends NACK for retransmission.

Algorithm:

1. Convert received bytes to soft bits (hard decision: 1 -> +127, 0 -> -127)
2. Pass soft bits to HARQ decoder (Chase Combining + Viterbi)
3. If decode succeeds: send ACK, reset combiner, return payload
4. If decode fails: send NACK, accumulate soft bits for next retransmit

Precondition

link->initialized == true (link must be initialized)
frame->payload contains FEC-encoded data (FEC flag set)
result->payload buffer >= k_harq_max_payload (1024 bytes minimum)
Single-threaded context (SPI receive path protected by state machine)

Postcondition

On success: result->fec_decoded == true, result->payload filled
On success: ACK sent to sender, HARQ combiner reset
On failure: NACK sent, soft bits accumulated in combiner
result->combining_count reflects number of transmissions combined

Note

Thread safety: This function is NOT thread-safe. Must be called from single-threaded SPI receive context (enforced by link state machine).

See also

internal_fec_decoded_len() FEC decoded length calculation

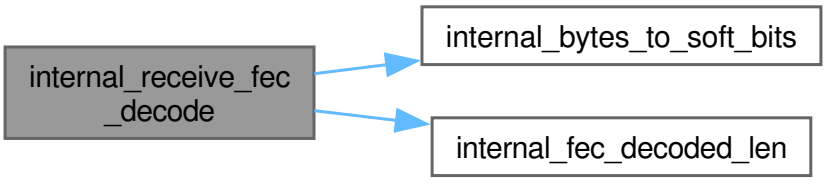
Definition at line 634 of file rx_spi_link.c.

```
00637 {
00638   /* Convert bytes to soft bits for Viterbi decoder.
00639    * NOTE: Using static buffer to avoid stack overflow (16,400 bytes).
00640    * Safe because this function is called from single-threaded context. */
00641   static rx_soft_bit_t s_soft_bits[k_harq_soft_buffer_size];
00642   uint32_t             soft_len = 0;
00643
00644   (void)internal_bytes_to_soft_bits(frame->payload,
00645                                     frame->header.length,
00646                                     s_soft_bits,
00647                                     k_harq_soft_buffer_size,
00648                                     &soft_len);
00649
00650   const rx_harq_decode_params_t params = {
00651     .soft_bits          = s_soft_bits,
00652     .soft_len           = soft_len,
00653     .expected_output_len = internal_fec_decoded_len(soft_len),
00654   };
00655
00656   const rx_err_t err = rx_harq_decode(&link->harq, &params, result->payload, &result->payload_len);
00657
00658   result->fec_decoded     = true;
00659   result->combining_count = rx_harq_get_retry_count(&link->harq);
00660
00661   if (err == k_rx_ok) {
00662     /* Decode success - send ACK */
00663     const rx_err_t ack_err = rx_spi_comm_send_ack(link->spi_handle, frame->header.sequence);
00664     if (ack_err != k_rx_ok) {
00665       rx_log_warn(s_tag, "ACK send failed (data decoded OK)");
00666     }
00667     (void)rx_harq_reset(&link->harq);
00668     return k_rx_ok;
00669   }
00670
00671   /* Decode failed - send NACK for retransmission */
00672   rx_log_debug(s_tag, "FEC decode failed, sending NACK");
00673   (void)rx_spi_comm_send_nack(link->spi_handle, frame->header.sequence, k_frame_flag_soft_nack);
00674   return k_rx_err_protocol_error;
00675 }
```

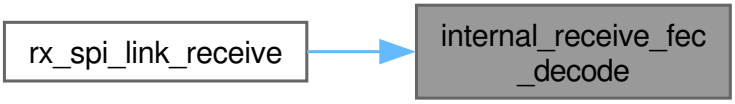
References rx_spi_link_receive_result_t::combining_count, rx_spi_link_receive_result_t::fec_decoded, rx_spi_link_t::harq, internal_bytes_to_soft_bits(), internal_fec_decoded_len(), rx_spi_link_receive_result_t::paylo rx_spi_link_receive_result_t::payload_len, s_tag, and rx_spi_link_t::spi_handle.

Referenced by rx_spi_link_receive().

Here is the call graph for this function:




Here is the caller graph for this function:



#### 4.3.3.5 internal_receive_passthrough()

```
RX_STATIC_TESTABLE rx_err_t internal_receive_passthrough (
                rx_spi_link_t * link,
                const rx_frame_t * frame,
                rx_spi_link_receive_result_t * result)
```

Process received frame without FEC (passthrough mode).

Copies payload directly to result buffer and sends ACK if required by frame flags. No FEC decoding or Chase Combining applied.

Definition at line 686 of file rx_spi_link.c.

```
00689 {
00690   /* No FEC: passthrough (copy payload directly). */
00691   if (frame->header.length > 0 && frame->header.length <= k_harq_max_payload) {
00692     for (uint16_t i = 0; i < frame->header.length; i++) {
00693       result->payload[i] = frame->payload[i];
00694     }
00695     result->payload_len = frame->header.length;
00696   } else if (frame->header.length > k_harq_max_payload) {
00697     /* Defensive: bound payload_len to buffer capacity to prevent out-of-bounds reads */
00698     rx_log_warn(s_tag, "Passthrough payload too large, truncating");
00699     for (uint32_t i = 0; i < k_harq_max_payload; i++) {
00700       result->payload[i] = frame->payload[i];
00701     }
00702     result->payload_len = k_harq_max_payload;
00703   } else {
00704     /* Zero-length payload */
00705     result->payload_len = 0;
00706   }
00707   result->fec_decoded     = false;
00708   result->combining_count = 1;
00709
00710   /* Send ACK if the frame requires it */
00711   if ((frame->header.flags & k_frame_flag_requires_ack) != 0) {
00712     const rx_err_t ack_err = rx_spi_comm_send_ack(link->spi_handle, frame->header.sequence);
00713     if (ack_err != k_rx_ok) {
00714       rx_log_warn(s_tag, "ACK send failed (passthrough)");
00715     }
00716   }
00717
00718   return k_rx_ok;
00719 }
```

References rx_spi_link_receive_result_t::combining_count, rx_spi_link_receive_result_t::fec_decoded, rx_spi_link_receive_result_t::payload, rx_spi_link_receive_result_t::payload_len, s_tag, and rx_spi_link_t::spi_handle.

Referenced by rx_spi_link_receive().

Here is the caller graph for this function:


#### 4.3.3.6 internal_send_attempt()

```
RX_STATIC_TESTABLE rx_err_t internal_send_attempt (
                rx_spi_link_t * link,
                rx_frame_type_t type,
                uint8_t base_flags,
                const uint8_t * tx_payload,
                uint32_t tx_len,
                uint8_t attempt)
```

Attempt a single transmission with ACK/NACK handling.

Sends frame via SPI transport, waits for ACK/NACK, and handles retransmission logic. Updates link state for retransmit/waiting_ack. Gets TX sequence BEFORE sending to avoid race conditions with external sequence management.

Note

Parameter ordering convention: type, base_flags, tx_payload, tx_len, attempt. This ordering groups frame metadata (type, flags) before payload data (payload, len), then retry state (attempt). While adjacent integer parameters may appear swappable, the semantic grouping is intentional for code clarity.

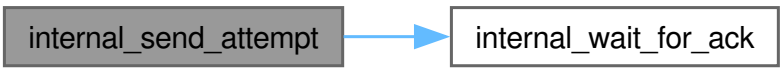
Definition at line 392 of file rx_spi_link.c.

```
00398 {
00399   /* Audit F-04: enforce the single-task invariant on the link send path.
00400    * atomic_flag_test_and_set returns the PREVIOUS value; if it was
00401    * already true, another task / ISR is mid-send and will race with us
00402    * on rx_session_get_tx() / rx_spi_comm_send(). Halt now -- the bug is
00403    * upstream (a missing mutex or the wrong task-priority assumption)
00404    * and continuing would corrupt the session sequence. */
00405   RX_ASSERT_PRE(!atomic_flag_test_and_set(&s_send_path_busy),
00406                 "rx_spi_link send-path re-entered: single-task invariant broken");
00407
00408   uint8_t flags = base_flags;
00409
00410   /* Mark retransmissions */
00411   if (attempt > 0) {
00412     flags |= k_frame_flag_retransmit;
00413     link->state = k_spi_link_state_retransmitting;
00414   } else {
00415     link->state = k_spi_link_state_waiting_ack;
00416   }
00417
00418   /* CRITICAL: Get TX sequence BEFORE sending to avoid race.
00419      * rx_session_get_tx() returns the NEXT sequence to be used.
00420      * rx_spi_comm_send() will use this sequence and then increment.
00421      * This approach is safe ONLY if rx_spi_link_send() is not called
00422      * concurrently (enforced by caller via external mutex or task priority).
00423      *
00424      * Alternative solution: Modify rx_spi_comm_send() to return the
00425      * sequence actually used, but that requires API change across codebase. */
00426   /* rx_session_get_tx only fails if session is uninitialized or mutex fails;
00427    * both are invariants that must hold by the time send is called. */
00428   uint16_t next_seq = 0;
00429   (void)(rx_session_get_tx(link->spi_handle->session, &next_seq));
00430   const uint16_t expected_seq = next_seq; /* This will be used by send */
00431
00432   rx_err_t result;
00433
00434   /* Send frame via SPI transport */
00435   const rx_err_t send_err = rx_spi_comm_send(link->spi_handle, type, flags, tx_payload, tx_len);
00436   if (send_err != k_rx_ok) {
00437     rx_log_warn(s_tag, "SPI send failed");
00438     result = send_err;
00439   } else {
00440     /* Wait for ACK/NACK using the sequence we captured */
00441     result = internal_wait_for_ack(link, expected_seq);
00442   }
00443
00444   /* Release the single-task guard before returning so subsequent
00445    * (sequential) calls succeed. Cleared regardless of result so a
00446    * single failure does not permanently lock the path. */
00447   atomic_flag_clear(&s_send_path_busy);
00448   return result;
00449 }
```

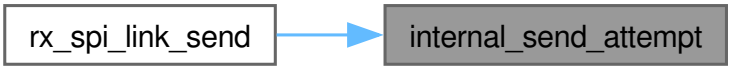
References internal_wait_for_ack(), k_spi_link_state_retransmitting, k_spi_link_state_waiting_ack, s_send_path_busy, s_tag, rx_spi_link_t::spi_handle, and rx_spi_link_t::state.

Referenced by rx_spi_link_send().

Here is the call graph for this function:




Here is the caller graph for this function:



#### 4.3.3.7 internal_wait_for_ack()

```
RX_STATIC_TESTABLE rx_err_t internal_wait_for_ack (
                rx_spi_link_t * link,
                uint16_t expected_seq)
```

Wait for ACK/NACK response from gateway.

Polls rx_spi_comm_receive() looking for an ACK or NACK frame matching the expected sequence number. Non-matching frames are logged and ignored.

Precondition

link must be initialized

expected_seq matches the last sent frame's sequence

Postcondition

No side effects on timeout

Since

Version 1.0.0

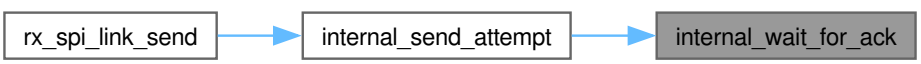
Definition at line 245 of file rx_spi_link.c.

```
00246 {
00247   rx_frame_t     ack_frame = {};
00248   const rx_err_t err = rx_spi_comm_receive(link->spi_handle, &ack_frame, k_spi_link_ack_timeout_ms);
00249
00250   if (err == k_rx_err_timeout) {
00251     return k_rx_err_timeout;
00252   }
00253   if (err != k_rx_ok) {
00254     rx_log_warn(s_tag, "ACK receive error");
00255     return err;
00256   }
00257
00258   /* Check frame type */
00259   if (ack_frame.header.type == k_frame_type_ack) {
00260     if (ack_frame.header.sequence == expected_seq) {
00261       return k_rx_ok;
00262     }
00263     rx_log_warn(s_tag, "ACK seq mismatch");
00264     return k_rx_err_timeout; /* Treat as timeout, will retry */
00265   }
00266
00267   if (ack_frame.header.type == k_frame_type_nack) {
00268     if (ack_frame.header.sequence == expected_seq) {
00269       return k_rx_err_protocol_error; /* NACK = decode failure on receiver */
00270     }
00271     rx_log_warn(s_tag, "NACK seq mismatch");
00272     return k_rx_err_timeout;
00273   }
00274
00275   /* Unexpected frame type (data frame while waiting for ACK) - ignore */
00276   rx_log_debug(s_tag, "Non-ACK frame during wait");
00277   return k_rx_err_timeout;
00278 }
```

References k_spi_link_ack_timeout_ms, s_tag, and rx_spi_link_t::spi_handle.

Referenced by internal_send_attempt().

Here is the caller graph for this function:



#### 4.3.3.8 rx_spi_link_deinit()

```
rx_err_t rx_spi_link_deinit (
                rx_spi_link_t * link)    [nodiscard]
```

Deinitialize SPI link layer.

See also

rx_spi_link.h for full documentation

Definition at line 349 of file rx_spi_link.c.

```
00350 {
00351   /* Pre-condition 1 */
00352   if (link == nullptr) {
00353     return k_rx_err_invalid_arg;
00354   }
00355
00356   /* Pre-condition 2 */
00357   if (!link->initialized) {
00358     return k_rx_err_invalid_state;
00359   }
00360
00361   /* Deinit HARQ subsystem. HARQ deinit only fails if FEC encoder/decoder
00362    * deinit fails, which cannot happen after a successful init on target. */
00363   (void)(rx_harq_deinit(&link->harq));
00364
00365   /* Post-condition: Mark uninitialized */
00366   link->initialized = false;
00367   link->state       = k_spi_link_state_idle;
00368
00369   rx_log_info(s_tag, "Deinitialized");
00370   return k_rx_ok;
00371 }
```

References rx_spi_link_t::harq, rx_spi_link_t::initialized, k_spi_link_state_idle, s_tag, and rx_spi_link_t::state.

#### 4.3.3.9 rx_spi_link_fec_enabled()

```
bool rx_spi_link_fec_enabled (
                const rx_spi_link_t * link)
```

Check if FEC is enabled on this link.

See also

rx_spi_link.h for full documentation

Definition at line 829 of file rx_spi_link.c.

```
00830 {
00831   if (link == nullptr || !link->initialized) {
00832     return false;
00833   }
00834   return link->fec_enabled;
00835 }
```

References rx_spi_link_t::fec_enabled, and rx_spi_link_t::initialized.

#### 4.3.3.10 rx_spi_link_get_state()

```
rx_spi_link_state_t rx_spi_link_get_state (
                const rx_spi_link_t * link)
```

Get current SPI link state.

See also

rx_spi_link.h for full documentation

Definition at line 817 of file rx_spi_link.c.

```
00818 {
00819   if (link == nullptr || !link->initialized) {
00820     return k_spi_link_state_error;
00821   }
00822   return link->state;
00823 }
```

References rx_spi_link_t::initialized, k_spi_link_state_error, and rx_spi_link_t::state.

#### 4.3.3.11 rx_spi_link_init()

```
rx_err_t rx_spi_link_init (
                rx_spi_link_t * link,
                const rx_spi_link_config_t * config)    [nodiscard]
```

Initialize SPI link layer.

See also

rx_spi_link.h for full documentation

Definition at line 300 of file rx_spi_link.c.

```
00301 {
00302   /* Pre-condition 1: NULL checks */
00303   if (link == nullptr || config == nullptr) {
00304     return k_rx_err_invalid_arg;
00305   }
00306
00307   /* Pre-condition 2: SPI handle required */
00308   if (config->spi_handle == nullptr) {
00309     rx_log_error(s_tag, "SPI handle is required");
00310     return k_rx_err_invalid_arg;
00311   }
00312
00313   *link = s_zero_link;
00314
00315   /* Store configuration */
00316   link->spi_handle  = config->spi_handle;
00317   link->fec_enabled = config->fec_enabled;
00318   link->max_retries =
00319     (config->max_retries > 0) ? config->max_retries : k_spi_link_default_max_retries;
00320
00321   /* Initialize HARQ handle (includes FEC encoder/decoder and Chase Combiner) */
00322   const rx_harq_config_t harq_cfg = {
00323     .max_retries  = link->max_retries,
00324     .fec_enabled  = (int)link->fec_enabled ? 1U : 0U,
00325     .max_combines = link->max_retries,
00326   };
00327
00328   /* HARQ init only fails if FEC encoder/decoder init fails, which cannot
00329    * happen with valid config on the target. This is a defensive assertion. */
00330   (void)(rx_harq_init(&link->harq, &harq_cfg));
00331
00332   /* Post-condition 1: Mark initialized */
00333   link->state       = k_spi_link_state_idle;
00334   link->initialized = true;
```

```
00335
00336   /* Post-condition 2: Verify HARQ is ready.
00337    * rx_harq_get_state returns idle after successful init; this is a
00338    * defensive invariant check. */
00339
00340   rx_log_info(s_tag, "Init OK");
00341
00342   return k_rx_ok;
00343 }
```

References rx_spi_link_config_t::fec_enabled, rx_spi_link_t::fec_enabled, rx_spi_link_t::harq, rx_spi_link_t::initialized, k_spi_link_default_max_retries, k_spi_link_state_idle, rx_spi_link_config_t::max_retrie rx_spi_link_t::max_retries, s_tag, s_zero_link, rx_spi_link_config_t::spi_handle, rx_spi_link_t::spi_handle, and rx_spi_link_t::state.

#### 4.3.3.12 rx_spi_link_receive()

```
rx_err_t rx_spi_link_receive (
                rx_spi_link_t * link,
                rx_spi_link_receive_result_t * result,
                uint32_t timeout_ms)   [nodiscard]
```

Receive and decode a frame from SPI with HARQ.

See also

rx_spi_link.h for full documentation

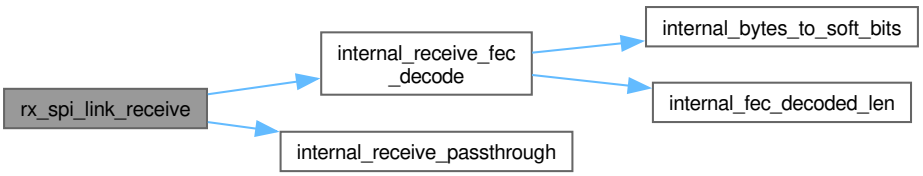
Definition at line 726 of file rx_spi_link.c.

```
00727 {
00728   /* Pre-condition 1: NULL checks */
00729   if (link == nullptr || result == nullptr) {
00730     return k_rx_err_invalid_arg;
00731   }
00732
00733   /* Pre-condition 2: State check */
00734   if (!link->initialized) {
00735     return k_rx_err_invalid_state;
00736   }
00737
00738   /* Receive raw frame from SPI transport */
00739   rx_frame_t frame = {};
00740   *result          = (rx_spi_link_receive_result_t){};
00741
00742   const rx_err_t recv_err = rx_spi_comm_receive(link->spi_handle, &frame, timeout_ms);
00743   if (recv_err != k_rx_ok) {
00744     return recv_err; /* Timeout or error */
00745   }
00746
00747   /* Handle ACK/NACK control frames internally (not data for application) */
00748   if (frame.header.type == k_frame_type_ack || frame.header.type == k_frame_type_nack) {
00749     /* Control frame - not application data */
00750     return k_rx_err_no_data; /* Signal "no data" to caller */
00751   }
00752
00753   /* Handle PING/PONG/RESET (already handled by rx_spi_comm internally) */
00754   const bool is_ping      = (bool)(frame.header.type == k_frame_type_ping);
00755   const bool is_pong      = (bool)(frame.header.type == k_frame_type_pong);
00756   const bool is_reset     = (bool)(frame.header.type == k_frame_type_reset);
00757   const bool is_reset_ack = (bool)(frame.header.type == k_frame_type_reset_ack);
00758   if ((bool)((int)is_ping | (int)is_pong | (int)is_reset | (int)is_reset_ack)) {
00759     return k_rx_err_no_data; /* Not application data */
00760   }
00761
00762   /* Data frame received - populate common metadata */
00763   result->sequence      = frame.header.sequence;
00764   result->frame_type    = frame.header.type;
00765   result->is_retransmit = (bool)((frame.header.flags & k_frame_flag_retransmit) != 0);
00766
00767   /* Check if FEC decoding is needed */
00768   const bool has_fec_flag = (bool)((frame.header.flags & k_frame_flag_fec_enabled) != 0);
00769
00770   const bool fec_enabled_flag = link->fec_enabled;
```

```
00771   const bool fec_frame_flag   = has_fec_flag;
00772   const bool has_payload      = (bool)(frame.header.length > 0);
00773   if ((bool)((int)fec_enabled_flag & (int)fec_frame_flag & (int)has_payload)) {
00774     /* FEC decode path: HARQ with Chase Combining */
00775     return internal_receive_fec_decode(link, &frame, result);
00776   }
00777
00778   /* No FEC: passthrough (copy payload directly) */
00779   return internal_receive_passthrough(link, &frame, result);
00780 }
```

References rx_spi_link_t::fec_enabled, rx_spi_link_receive_result_t::frame_type, rx_spi_link_t::initialized, internal_receive_fec_decode(), internal_receive_passthrough(), rx_spi_link_receive_result_t::is_retransmit, rx_spi_link_receive_result_t::sequence, and rx_spi_link_t::spi_handle.

Here is the call graph for this function:




#### 4.3.3.13 rx_spi_link_reset()

```
rx_err_t rx_spi_link_reset (
                rx_spi_link_t * link)   [nodiscard]
```

Reset SPI link layer for new session.

See also

rx_spi_link.h for full documentation

Definition at line 790 of file rx_spi_link.c.

```
00791 {
00792   /* Pre-condition 1 */
00793   if (link == nullptr) {
00794     return k_rx_err_invalid_arg;
00795   }
00796
00797   /* Pre-condition 2 */
00798   if (!link->initialized) {
00799     return k_rx_err_invalid_state;
00800   }
00801
00802   /* Reset HARQ (clears combiner and retry count). Only fails if harq is
00803    * null/uninitialized -- both prevented by link->initialized check above. */
00804   (void)(rx_harq_reset(&link->harq));
00805
00806   /* Post-condition: return to idle */
00807   link->state = k_spi_link_state_idle;
00808
00809   rx_log_debug(s_tag, "Link reset");
00810   return k_rx_ok;
00811 }
```

References rx_spi_link_t::harq, rx_spi_link_t::initialized, k_spi_link_state_idle, s_tag, and rx_spi_link_t::state.

#### 4.3.3.14 rx_spi_link_send()

```
rx_err_t rx_spi_link_send (
                rx_spi_link_t * link,
                rx_frame_type_t type,
                const uint8_t * payload,
                uint32_t payload_len)   [nodiscard]
```

Send payload with HARQ reliability over SPI.

See also

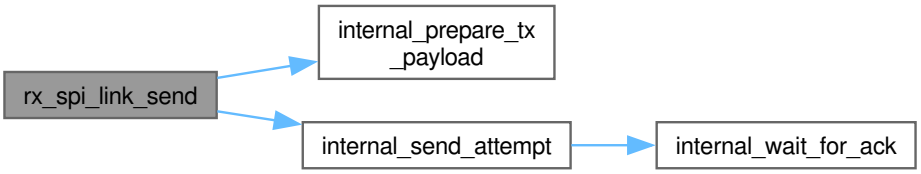
rx_spi_link.h for full documentation

Definition at line 501 of file rx_spi_link.c.

```
00505 {
00506   /* Pre-condition 1: NULL and state checks */
00507   if (link == nullptr) {
00508     return k_rx_err_invalid_arg;
00509   }
00510   if (!link->initialized) {
00511     return k_rx_err_invalid_state;
00512   }
00513
00514   /* Pre-condition 2: Payload validation */
00515   if (payload == nullptr && payload_len > 0) {
00516     return k_rx_err_invalid_arg;
00517   }
00518   if (payload_len > k_harq_max_payload) {
00519     rx_log_error(s_tag, "Payload too large for HARQ");
00520     return k_rx_err_invalid_size;
00521   }
00522
00523   /* Reset HARQ for new transaction. Fails only if harq is null/uninitialized;
00524    * both are invariants checked above via link->initialized. */
00525   (void)rx_harq_reset(&link->harq);
00526
00527   /* Prepare payload: FEC encode if enabled, otherwise passthrough.
00528    * internal_prepare_tx_payload only fails if FEC encode fails, which is
00529    * prevented by the payload_len <= k_harq_max_payload check above. */
00530   const uint8_t* tx_payload = nullptr;
00531   uint32_t       tx_len     = 0;
00532   uint8_t        base_flags = 0;
00533
00534   (void)internal_prepare_tx_payload(link, payload, payload_len, &tx_payload, &tx_len, &base_flags);
00535
00536   /* Retry loop (bounded by max_retries)
00537      * NOTE: This function is NOT thread-safe. Caller must ensure that
00538      * rx_spi_link_send() is not called concurrently on the same link handle.
00539      * Typical enforcement: Single task owns link, or external mutex protection. */
00540   for (uint8_t attempt = 0; attempt < link->max_retries; attempt++) {
00541     const rx_err_t err = internal_send_attempt(link, type, base_flags, tx_payload, tx_len, attempt);
00542
00543     if (err == k_rx_ok) {
00544       /* ACK received - success */
00545       link->state = k_spi_link_state_idle;
00546       return k_rx_ok;
00547     }
00548
00549     if (err == k_rx_err_protocol_error) {
00550       /* NACK received - gateway decode failed, retry */
00551       rx_log_debug(s_tag, "NACK received, retrying");
00552       continue;
00553     }
00554
00555     /* Timeout or send error - retry */
00556     rx_log_debug(s_tag, "Send attempt failed, retrying");
00557   }
00558
00559   /* All retries exhausted */
00560   link->state = k_spi_link_state_error;
00561   rx_log_error(s_tag, "Max retries exceeded");
00562   return k_rx_err_retry_limit;
00563 }
```

References rx_spi_link_t::harq, rx_spi_link_t::initialized, internal_prepare_tx_payload(), internal_send_attempt(), k_spi_link_state_error, k_spi_link_state_idle, rx_spi_link_t::max_retries, s_tag, and rx_spi_link_t::state.

Here is the call graph for this function:




### 4.3.4 Variable Documentation

#### 4.3.4.1 s_send_path_busy

RX_STATIC_TESTABLE atomic_flag s_send_path_busy = ATOMIC_FLAG_INIT

Definition at line 120 of file rx_spi_link.c.

Referenced by internal_send_attempt().

#### 4.3.4.2 s_tag

const char s_tag[] = "SPI_LINK" [static]

Log tag for SPI link layer messages.

Definition at line 130 of file rx_spi_link.c.

Referenced by internal_receive_fec_decode(), internal_receive_passthrough(), internal_send_attempt(), internal_wait_for_ack(), rx_spi_link_deinit(), rx_spi_link_init(), rx_spi_link_reset(), and rx_spi_link_send().

#### 4.3.4.3 s_zero_link

const rx_spi_link_t s_zero_link = {} [static]

Zero-initialized SPI link template for stack-safe initialization.

Static const instance used to clear rx_spi_link_t without creating a large compound literal on the stack. Lives in .rodata section.

Note

Read-only; never modified after static initialization

Warning

Do not remove - prevents stack overflow in init function

See also

rx_spi_link_init() Uses this template to zero-initialize the link handle

Since

Version 1.0.0

Definition at line 294 of file rx_spi_link.c.

```
00294 {};
```

Referenced by rx_spi_link_init().

# 4.4 rx_spi_link.c

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_spi_link.c
00003  * @brief SPI Link Layer with HARQ Implementation
00004  *
00005  * @details
00006  * Implements the SPI link layer that sits between the communication manager
00007  * and the raw SPI transport (rx_spi_comm). This is a C port of the Go
00008  * gateway's `internal/link/spi.go`, providing:
00009  *
00010  * - FEC encoding on TX (convolutional K=7, rate 1/2)
00011  * - Soft-decision Viterbi decoding on RX
00012  * - Chase Combining across retransmissions
00013  * - ACK/NACK handshake with configurable retries
00014  *
00015  * ## Key Design Decisions
00016  *
00017  * 1. **Hard-to-soft conversion**: SPI is digital (no analog front-end), so
00018  *    received bytes are converted to hard soft bits: 1 -> +127, 0 -> -127.
00019  *    Chase Combining still provides gain because noise corrupts different
00020  *    bits on each retransmission.
00021  *
00022  * 2. **Blocking send**: rx_spi_link_send() blocks until ACK or all retries
00023  *    are exhausted. The comm_task runs at 100 Hz, giving ~10 ms per cycle.
00024  *    With k_spi_link_ack_timeout_ms = 10 ms and 3 retries, worst case is
00025  *    ~30 ms (occupies 3 comm cycles).
00026  *
00027  * 3. **Non-blocking receive**: rx_spi_link_receive() defers to
00028  *    rx_spi_comm_receive() with the caller's timeout. Control frames
00029  *    (ACK/NACK) are handled internally and not returned to the caller.
00030  *
00031  * ## Protocol Flow (TX)
00032  *
00033  * ```
00034  * payload
00035  *   |
00036  *   +--> rx_harq_encode() --> encoded_payload (2N+2 bytes if FEC)
00037  *   |                         |
00038  *   +--> (passthrough if no FEC)
00039  *   |
00040  *   v
00041  * for attempt = 1..3:
00042  *   rx_spi_comm_send(encoded_payload, REQUIRES_ACK | FEC_ENABLED)
00043  *   wait for ACK/NACK (10 ms timeout)
00044  *   ACK -> return success
00045  *   NACK/timeout -> set RETRANSMIT flag, retry
00046  * ```
00047  *
00048  * ## Protocol Flow (RX)
00049  *
00050  * ```
00051  * rx_spi_comm_receive()
00052  *   |
00053  *   +--> ACK/NACK -> dispatch internally, return "no data"
00054  *   |
00055  *   +--> data frame (FEC flag set):
00056  *   |      convert bytes to soft bits
00057  *   |      rx_harq_decode() (with Chase Combining)
00058  *   |      success -> send ACK, return decoded payload
00059  *   |      failure -> send NACK, return error
00060  *   |
00061  *   +--> data frame (no FEC flag):
00062  *          passthrough, send ACK, return payload
00063  * ```
00064  *
00065  * @par NASA Power of 10 Compliance
00066  * - Rule 1: [OK] No goto, setjmp/longjmp, recursion
00067  * - Rule 2: [OK] All loops bounded (max_retries, k_rx_bits_per_byte)
00068  * - Rule 3: [OK] Zero dynamic allocation
00069  * - Rule 4: [OK] All functions < 60 lines
00070  * - Rule 5: [OK] Minimum 2 preconditions per function
00071  * - Rule 6: [OK] Variables at smallest scope
00072  * - Rule 7: [OK] All return values checked
00073  * - Rule 8: [OK] C23 typed enums for all constants
00074  * - Rule 10: [OK] Compiled with -Wall -Wextra -Werror
00075  *
00076  * @see rx_spi_link.h  Public API
00077  * @see star-gateway/internal/link/spi.go  Go reference implementation
00078  *
00079  * @author Locked, Inc.
00080  * @date 2026-02-14
00081  * @copyright Copyright (c) 2026 Locked Inc.
00082  * SPDX-License-Identifier: MIT
```

```
00083  *
00084  * @since Version 1.0.0
00085  */
00086
00087 #include "rx_spi_link.h"
00088
00089 #include <stdatomic.h>
00090 #include <stdbool.h>
00091 #include <string.h>
00092
00093 #include "rx_check.h"
00094 #include "rx_fec.h"
00095 #include "rx_log.h"
00096
00097 /*
      ============================================================================
00098  * Audit F-04: single-task invariant guard for the link send path
00099  *
      ============================================================================
00100  *
00101  * The "get-then-send-then-increment" sequence-capture pattern in
00102  * internal_send_attempt() (rx_session_get_tx() captures the next
00103  * sequence, then rx_spi_comm_send() consumes and increments it) is
00104  * only race-free if exactly one task is in the link send path at a
00105  * time. The invariant is documented but was not enforced.
00106  *
00107  * s_send_path_busy is a process-wide atomic flag. The first entrant
00108  * sets it; a re-entrant caller observes the prior set and trips
00109  * RX_ASSERT_PRE, halting the system before it can corrupt the session
00110  * sequence. Cleared on exit.
00111  *
00112  * Use atomic_flag (lock-free, MMIO-safe on RXv3) rather than a plain
00113  * bool: even on a single-core MCU, an interrupt could pre-empt a task
00114  * mid-send. atomic_flag_test_and_set is the cheapest correct primitive.
00115  */
00116 /* RX_STATIC_TESTABLE: exposed in UNIT_TEST builds so tests can pre-set the
00117  * flag and reach the false branch of the RX_ASSERT_PRE in internal_send_attempt
00118  * (the re-entrancy guard's failure branch). Same testability pattern as the
00119  * RX_STATIC_TESTABLE function-qualifier already used in this file. */
00120 RX_STATIC_TESTABLE atomic_flag s_send_path_busy = ATOMIC_FLAG_INIT;
00121
00122 /*
      ============================================================================
00123  * Module Constants
00124  *
      ============================================================================
      */
00125
00126 /**
00127  * @var s_tag
00128  * @brief Log tag for SPI link layer messages
00129  */
00130 static const char s_tag[] = "SPI_LINK";
00131
00132 /**
00133  * @enum internal_soft_bit_values_t
00134  * @brief Soft bit conversion constants
00135  *
00136  * @details
00137  * Used when converting received hard bytes to soft bits for the Viterbi
00138  * decoder. Each bit becomes +127 (confident 1) or -127 (confident 0).
00139  * Matches Go's bytesToSoftBits() in internal/link/spi.go.
00140  *
00141  * @since Version 1.0.0
00142  */
00143 typedef enum : int8_t {
00144   k_internal_soft_bit_one  = 127,  /**< Hard bit 1 -> soft confidence +127 */
00145   k_internal_soft_bit_zero = -127, /**< Hard bit 0 -> soft confidence -127 */
00146 } internal_soft_bit_values_t;
00147
00148 /**
00149  * @enum internal_bit_constants_t
00150  * @brief Bit manipulation constants for byte-to-soft-bit conversion
00151  *
00152  * @since Version 1.0.0
00153  */
00154 typedef enum : uint8_t {
00155   k_spi_link_bits_per_byte = 8,    /**< Number of bits in a byte */
00156   k_spi_link_msb_shift     = 7,    /**< Shift to extract MSB */
00157   k_spi_link_bit_mask      = 0x01, /**< Mask to extract single bit */
00158 } internal_bit_constants_t;
00159
00160 /*
      ============================================================================
00161  * Internal Helpers (static)
00162  *
      ============================================================================
```

```
     */
00163
00164 /**
00165  * @brief Convert payload bytes to soft bits for Viterbi decoder
00166  *
00167  * @details
00168  * Each byte is expanded to 8 soft bits, MSB-first. Bit value 1 becomes
00169  * +127, bit value 0 becomes -127. This matches the Go gateway's
00170  * bytesToSoftBits() function.
00171  *
00172  *
00173  *
00174  * @pre data and soft must be non-NULL
00175  * @pre soft_size >= data_len * k_spi_link_bits_per_byte
00176  * @post soft_len == data_len * k_spi_link_bits_per_byte
00177  *
00178  * @since Version 1.0.0
00179  */
00180 RX_STATIC_TESTABLE rx_err_t internal_bytes_to_soft_bits(const uint8_t* data,
00181                                                         uint32_t       data_len,
00182                                                         rx_soft_bit_t* soft,
00183                                                         uint32_t       soft_size,
00184                                                         uint32_t*      soft_len)
00185 {
00186   /* Rule 5: Precondition validation - nullptr checks */
00187   if (data == nullptr) {
00188     if (soft_len != nullptr) {
00189       *soft_len = 0;
00190     }
00191     return k_rx_err_invalid_arg;
00192   }
00193
00194   if (soft == nullptr) {
00195     if (soft_len != nullptr) {
00196       *soft_len = 0;
00197     }
00198     return k_rx_err_invalid_arg;
00199   }
00200
00201   if (soft_len == nullptr) {
00202     return k_rx_err_invalid_arg;
00203   }
00204
00205   /* Rule 5: Precondition validation - overflow check before multiplication.
00206    * data_len > UINT32_MAX/8 means > 536 MB; this is an invariant the caller
00207    * must never violate (protocol enforces max payload of 1024 bytes). */
00208
00209   /* Rule 5: Precondition validation - buffer size check */
00210   const uint32_t required = data_len * (uint32_t)k_spi_link_bits_per_byte;
00211   if (required > soft_size) {
00212     *soft_len = 0;
00213     return k_rx_err_invalid_size;
00214   }
00215
00216   for (uint32_t byte_idx = 0; byte_idx < data_len; byte_idx++) {
00217     const uint8_t b = data[byte_idx];
00218     for (uint8_t bit_idx = 0; bit_idx < k_spi_link_bits_per_byte; bit_idx++) {
00219       const uint8_t  bit = (b >> (k_spi_link_msb_shift - bit_idx)) & k_spi_link_bit_mask;
00220       const uint32_t idx = byte_idx * (uint32_t)k_spi_link_bits_per_byte + bit_idx;
00221       soft[idx]          = (bit == k_spi_link_bit_mask) ? (rx_soft_bit_t)k_internal_soft_bit_one
00222                                                         : (rx_soft_bit_t)k_internal_soft_bit_zero;
00223     }
00224   }
00225
00226   *soft_len = required;
00227   return k_rx_ok;
00228 }
00229
00230 /**
00231  * @brief Wait for ACK/NACK response from gateway
00232  *
00233  * @details
00234  * Polls rx_spi_comm_receive() looking for an ACK or NACK frame matching
00235  * the expected sequence number. Non-matching frames are logged and ignored.
00236  *
00237  *
00238  *
00239  * @pre link must be initialized
00240  * @pre expected_seq matches the last sent frame's sequence
00241  * @post No side effects on timeout
00242  *
00243  * @since Version 1.0.0
00244  */
00245 RX_STATIC_TESTABLE rx_err_t internal_wait_for_ack(rx_spi_link_t* link, uint16_t expected_seq)
00246 {
00247   rx_frame_t     ack_frame = {};
00248   const rx_err_t err = rx_spi_comm_receive(link->spi_handle, &ack_frame, k_spi_link_ack_timeout_ms);
```

```
00249
00250   if (err == k_rx_err_timeout) {
00251     return k_rx_err_timeout;
00252   }
00253   if (err != k_rx_ok) {
00254     rx_log_warn(s_tag, "ACK receive error");
00255     return err;
00256   }
00257
00258   /* Check frame type */
00259   if (ack_frame.header.type == k_frame_type_ack) {
00260     if (ack_frame.header.sequence == expected_seq) {
00261       return k_rx_ok;
00262     }
00263     rx_log_warn(s_tag, "ACK seq mismatch");
00264     return k_rx_err_timeout; /* Treat as timeout, will retry */
00265   }
00266
00267   if (ack_frame.header.type == k_frame_type_nack) {
00268     if (ack_frame.header.sequence == expected_seq) {
00269       return k_rx_err_protocol_error; /* NACK = decode failure on receiver */
00270     }
00271     rx_log_warn(s_tag, "NACK seq mismatch");
00272     return k_rx_err_timeout;
00273   }
00274
00275   /* Unexpected frame type (data frame while waiting for ACK) - ignore */
00276   rx_log_debug(s_tag, "Non-ACK frame during wait");
00277   return k_rx_err_timeout;
00278 }
00279
00280 /*
     ============================================================================
00281  * Lifecycle API Implementation
00282  *
     ============================================================================
     */
00283
00284 /**
00285  * @var s_zero_link
00286  * @brief Zero-initialized SPI link template for stack-safe initialization
00287  * @details Static const instance used to clear rx_spi_link_t without creating
00288  *          a large compound literal on the stack. Lives in .rodata section.
00289  * @note Read-only; never modified after static initialization
00290  * @warning Do not remove - prevents stack overflow in init function
00291  * @see rx_spi_link_init() Uses this template to zero-initialize the link handle
00292  * @since Version 1.0.0
00293  */
00294 static const rx_spi_link_t s_zero_link = {};
00295
00296 /**
00297  * @brief Initialize SPI link layer
00298  * @see rx_spi_link.h for full documentation
00299  */
00300 rx_err_t rx_spi_link_init(rx_spi_link_t* link, const rx_spi_link_config_t* config)
00301 {
00302   /* Pre-condition 1: NULL checks */
00303   if (link == nullptr || config == nullptr) {
00304     return k_rx_err_invalid_arg;
00305   }
00306
00307   /* Pre-condition 2: SPI handle required */
00308   if (config->spi_handle == nullptr) {
00309     rx_log_error(s_tag, "SPI handle is required");
00310     return k_rx_err_invalid_arg;
00311   }
00312
00313   *link = s_zero_link;
00314
00315   /* Store configuration */
00316   link->spi_handle  = config->spi_handle;
00317   link->fec_enabled = config->fec_enabled;
00318   link->max_retries =
00319     (config->max_retries > 0) ? config->max_retries : k_spi_link_default_max_retries;
00320
00321   /* Initialize HARQ handle (includes FEC encoder/decoder and Chase Combiner) */
00322   const rx_harq_config_t harq_cfg = {
00323     .max_retries  = link->max_retries,
00324     .fec_enabled  = (int)link->fec_enabled ? 1U : 0U,
00325     .max_combines = link->max_retries,
00326   };
00327
00328   /* HARQ init only fails if FEC encoder/decoder init fails, which cannot
00329    * happen with valid config on the target. This is a defensive assertion. */
00330   (void)(rx_harq_init(&link->harq, &harq_cfg));
00331
00332   /* Post-condition 1: Mark initialized */
```

```
00333   link->state       = k_spi_link_state_idle;
00334   link->initialized = true;
00335
00336   /* Post-condition 2: Verify HARQ is ready.
00337    * rx_harq_get_state returns idle after successful init; this is a
00338    * defensive invariant check. */
00339
00340   rx_log_info(s_tag, "Init OK");
00341
00342   return k_rx_ok;
00343 }
00344
00345 /**
00346  * @brief Deinitialize SPI link layer
00347  * @see rx_spi_link.h for full documentation
00348  */
00349 rx_err_t rx_spi_link_deinit(rx_spi_link_t* link)
00350 {
00351   /* Pre-condition 1 */
00352   if (link == nullptr) {
00353     return k_rx_err_invalid_arg;
00354   }
00355
00356   /* Pre-condition 2 */
00357   if (!link->initialized) {
00358     return k_rx_err_invalid_state;
00359   }
00360
00361   /* Deinit HARQ subsystem. HARQ deinit only fails if FEC encoder/decoder
00362    * deinit fails, which cannot happen after a successful init on target. */
00363   (void)(rx_harq_deinit(&link->harq));
00364
00365   /* Post-condition: Mark uninitialized */
00366   link->initialized = false;
00367   link->state       = k_spi_link_state_idle;
00368
00369   rx_log_info(s_tag, "Deinitialized");
00370   return k_rx_ok;
00371 }
00372
00373 /*
      ============================================================================
00374  * Send API Implementation
00375  *
      ============================================================================
      */
00376
00377 /**
00378  * @brief Attempt a single transmission with ACK/NACK handling
00379  *
00380  * @details
00381  * Sends frame via SPI transport, waits for ACK/NACK, and handles retransmission
00382  * logic. Updates link state for retransmit/waiting_ack. Gets TX sequence BEFORE
00383  * sending to avoid race conditions with external sequence management.
00384  *
00385  * @note Parameter ordering convention: type, base_flags, tx_payload, tx_len, attempt.
00386  * This ordering groups frame metadata (type, flags) before payload data (payload, len),
00387  * then retry state (attempt). While adjacent integer parameters may appear swappable,
00388  * the semantic grouping is intentional for code clarity.
00389  *
00390  *
00391  */
00392 RX_STATIC_TESTABLE rx_err_t internal_send_attempt(rx_spi_link_t*  link,
00393                                                   rx_frame_type_t type,
00394                                                   uint8_t         base_flags,
00395                                                   const uint8_t*  tx_payload,
00396                                                   uint32_t        tx_len,
00397                                                   uint8_t         attempt)
00398 {
00399   /* Audit F-04: enforce the single-task invariant on the link send path.
00400    * atomic_flag_test_and_set returns the PREVIOUS value; if it was
00401    * already true, another task / ISR is mid-send and will race with us
00402    * on rx_session_get_tx() / rx_spi_comm_send(). Halt now -- the bug is
00403    * upstream (a missing mutex or the wrong task-priority assumption)
00404    * and continuing would corrupt the session sequence. */
00405   RX_ASSERT_PRE(!atomic_flag_test_and_set(&s_send_path_busy),
00406                 "rx_spi_link send-path re-entered: single-task invariant broken");
00407
00408   uint8_t flags = base_flags;
00409
00410   /* Mark retransmissions */
00411   if (attempt > 0) {
00412     flags |= k_frame_flag_retransmit;
00413     link->state = k_spi_link_state_retransmitting;
00414   } else {
00415     link->state = k_spi_link_state_waiting_ack;
00416   }
```

```
00417
00418   /* CRITICAL: Get TX sequence BEFORE sending to avoid race.
00419      * rx_session_get_tx() returns the NEXT sequence to be used.
00420      * rx_spi_comm_send() will use this sequence and then increment.
00421      * This approach is safe ONLY if rx_spi_link_send() is not called
00422      * concurrently (enforced by caller via external mutex or task priority).
00423      *
00424      * Alternative solution: Modify rx_spi_comm_send() to return the
00425      * sequence actually used, but that requires API change across codebase. */
00426   /* rx_session_get_tx only fails if session is uninitialized or mutex fails;
00427    * both are invariants that must hold by the time send is called. */
00428   uint16_t next_seq = 0;
00429   (void)(rx_session_get_tx(link->spi_handle->session, &next_seq));
00430   const uint16_t expected_seq = next_seq; /* This will be used by send */
00431
00432   rx_err_t result;
00433
00434   /* Send frame via SPI transport */
00435   const rx_err_t send_err = rx_spi_comm_send(link->spi_handle, type, flags, tx_payload, tx_len);
00436   if (send_err != k_rx_ok) {
00437     rx_log_warn(s_tag, "SPI send failed");
00438     result = send_err;
00439   } else {
00440     /* Wait for ACK/NACK using the sequence we captured */
00441     result = internal_wait_for_ack(link, expected_seq);
00442   }
00443
00444   /* Release the single-task guard before returning so subsequent
00445    * (sequential) calls succeed. Cleared regardless of result so a
00446    * single failure does not permanently lock the path. */
00447   atomic_flag_clear(&s_send_path_busy);
00448   return result;
00449 }
00450
00451 /**
00452  * @brief Prepare payload for transmission (FEC encode or passthrough)
00453  *
00454  * @details
00455  * If FEC is enabled and payload is non-empty, encodes via rx_harq_encode()
00456  * and sets FEC flag. Otherwise, passes payload through unchanged. Sets
00457  * base_flags to include k_frame_flag_requires_ack and optionally
00458  * k_frame_flag_fec_enabled.
00459  *
00460  *
00461  *
00462  * @pre link->fec_enabled indicates whether FEC should be used
00463  * @pre link->fec_encode_buf sized for k_spi_link_max_encoded_payload
00464  * @post If FEC enabled: out_payload points to link->fec_encode_buf, out_flags has FEC flag
00465  * @post If FEC disabled: out_payload == payload, out_len == payload_len
00466  */
00467 RX_STATIC_TESTABLE rx_err_t internal_prepare_tx_payload(rx_spi_link_t*  link,
00468                                                         const uint8_t*  payload,
00469                                                         uint32_t        payload_len,
00470                                                         const uint8_t** out_payload,
00471                                                         uint32_t*       out_len,
00472                                                         uint8_t*        out_flags)
00473 {
00474   *out_payload = payload;
00475   *out_len     = payload_len;
00476   *out_flags   = k_frame_flag_requires_ack;
00477
00478   if ((int)link->fec_enabled && payload != nullptr && payload_len > 0) {
00479     uint32_t encoded_len = 0;
00480     (void)(rx_harq_encode(&link->harq,
00481                           payload,
00482                           payload_len,
00483                           link->fec_encode_buf,
00484                           (uint32_t)k_spi_link_max_encoded_payload,
00485                           &encoded_len));
00486     /* rx_harq_encode only fails if payload > k_harq_max_payload or harq
00487      * uninitialized -- both are invariants checked before this call. */
00488
00489     *out_payload = link->fec_encode_buf;
00490     *out_len     = encoded_len;
00491     *out_flags |= k_frame_flag_fec_enabled;
00492   }
00493
00494   return k_rx_ok;
00495 }
00496
00497 /**
00498  * @brief Send payload with HARQ reliability over SPI
00499  * @see rx_spi_link.h for full documentation
00500  */
00501 rx_err_t rx_spi_link_send(rx_spi_link_t*  link,
00502                           rx_frame_type_t type,
00503                           const uint8_t*  payload,
```

```
00504                           uint32_t        payload_len)
00505 {
00506   /* Pre-condition 1: NULL and state checks */
00507   if (link == nullptr) {
00508     return k_rx_err_invalid_arg;
00509   }
00510   if (!link->initialized) {
00511     return k_rx_err_invalid_state;
00512   }
00513
00514   /* Pre-condition 2: Payload validation */
00515   if (payload == nullptr && payload_len > 0) {
00516     return k_rx_err_invalid_arg;
00517   }
00518   if (payload_len > k_harq_max_payload) {
00519     rx_log_error(s_tag, "Payload too large for HARQ");
00520     return k_rx_err_invalid_size;
00521   }
00522
00523   /* Reset HARQ for new transaction. Fails only if harq is null/uninitialized;
00524    * both are invariants checked above via link->initialized. */
00525   (void)rx_harq_reset(&link->harq);
00526
00527   /* Prepare payload: FEC encode if enabled, otherwise passthrough.
00528    * internal_prepare_tx_payload only fails if FEC encode fails, which is
00529    * prevented by the payload_len <= k_harq_max_payload check above. */
00530   const uint8_t* tx_payload = nullptr;
00531   uint32_t       tx_len     = 0;
00532   uint8_t        base_flags = 0;
00533
00534   (void)internal_prepare_tx_payload(link, payload, payload_len, &tx_payload, &tx_len, &base_flags);
00535
00536   /* Retry loop (bounded by max_retries)
00537      * NOTE: This function is NOT thread-safe. Caller must ensure that
00538      * rx_spi_link_send() is not called concurrently on the same link handle.
00539      * Typical enforcement: Single task owns link, or external mutex protection. */
00540   for (uint8_t attempt = 0; attempt < link->max_retries; attempt++) {
00541     const rx_err_t err = internal_send_attempt(link, type, base_flags, tx_payload, tx_len, attempt);
00542
00543     if (err == k_rx_ok) {
00544       /* ACK received - success */
00545       link->state = k_spi_link_state_idle;
00546       return k_rx_ok;
00547     }
00548
00549     if (err == k_rx_err_protocol_error) {
00550       /* NACK received - gateway decode failed, retry */
00551       rx_log_debug(s_tag, "NACK received, retrying");
00552       continue;
00553     }
00554
00555     /* Timeout or send error - retry */
00556     rx_log_debug(s_tag, "Send attempt failed, retrying");
00557   }
00558
00559   /* All retries exhausted */
00560   link->state = k_spi_link_state_error;
00561   rx_log_error(s_tag, "Max retries exceeded");
00562   return k_rx_err_retry_limit;
00563 }
00564
00565 /*
      ============================================================================
00566  * Receive API Implementation
00567  *
      ============================================================================
      */
00568
00569 /**
00570  * @brief Calculate expected decoded payload length from FEC soft bit count
00571  *
00572  * @details
00573  * The FEC encoder packs output bits into bytes, rounding up to a whole byte.
00574  * For N input bytes the encoder produces:
00575  *   total_output_bits = (N*8 + k_fec_tail_bits) * k_fec_num_outputs
00576  *   encoded_bytes     = ceil(total_output_bits / 8)
00577  *
00578  * After internal_bytes_to_soft_bits():
00579  *   soft_len = encoded_bytes * 8  (may include up to 7 padding bits)
00580  *
00581  * Working backwards:
00582  *   num_symbols (including tail) = soft_len / k_fec_num_outputs
00583  *   input_bits  (without tail)   = num_symbols - k_fec_tail_bits
00584  *   expected_decoded_len         = input_bits / k_spi_link_bits_per_byte
00585  *
00586  * Note: integer division truncates, which is correct because the padding
00587  * bits in the last encoded byte are zero and do not contribute to symbols.
```

```
00588  *
00589  *
00590  *
00591  * @pre soft_len is the output of internal_bytes_to_soft_bits()
00592  * @pre soft_len >= k_fec_num_outputs * k_fec_tail_bits (else underflow)
00593  * @post Return value represents the original pre-FEC payload size
00594  *
00595  * @since Version 1.0.0
00596  */
00597 static inline uint32_t internal_fec_decoded_len(uint32_t soft_len)
00598 {
00599   return (soft_len / (uint32_t)k_fec_num_outputs - (uint32_t)k_fec_tail_bits) /
00600          (uint32_t)k_spi_link_bits_per_byte;
00601 }
00602
00603 /**
00604  * @brief Process received frame with FEC decoding and Chase Combining
00605  *
00606  * @details
00607  * Converts hard bits to soft bits, performs Viterbi decoding via HARQ,
00608  * and sends ACK/NACK based on decode success. On success, resets HARQ
00609  * combiner. On failure, sends NACK for retransmission.
00610  *
00611  * Algorithm:
00612  * 1. Convert received bytes to soft bits (hard decision: 1 -> +127, 0 -> -127)
00613  * 2. Pass soft bits to HARQ decoder (Chase Combining + Viterbi)
00614  * 3. If decode succeeds: send ACK, reset combiner, return payload
00615  * 4. If decode fails: send NACK, accumulate soft bits for next retransmit
00616  *
00617  *
00618  *
00619  * @pre link->initialized == true (link must be initialized)
00620  * @pre frame->payload contains FEC-encoded data (FEC flag set)
00621  * @pre result->payload buffer >= k_harq_max_payload (1024 bytes minimum)
00622  * @pre Single-threaded context (SPI receive path protected by state machine)
00623  *
00624  * @post On success: result->fec_decoded == true, result->payload filled
00625  * @post On success: ACK sent to sender, HARQ combiner reset
00626  * @post On failure: NACK sent, soft bits accumulated in combiner
00627  * @post result->combining_count reflects number of transmissions combined
00628  *
00629  * @note Thread safety: This function is NOT thread-safe. Must be called from
00630  * single-threaded SPI receive context (enforced by link state machine).
00631  *
00632  * @see internal_fec_decoded_len() FEC decoded length calculation
00633  */
00634 RX_STATIC_TESTABLE rx_err_t internal_receive_fec_decode(rx_spi_link_t*              link,
00635                                                         const rx_frame_t*           frame,
00636                                                         rx_spi_link_receive_result_t* result)
00637 {
00638   /* Convert bytes to soft bits for Viterbi decoder.
00639    * NOTE: Using static buffer to avoid stack overflow (16,400 bytes).
00640    * Safe because this function is called from single-threaded context. */
00641   static rx_soft_bit_t s_soft_bits[k_harq_soft_buffer_size];
00642   uint32_t             soft_len = 0;
00643
00644   (void)internal_bytes_to_soft_bits(frame->payload,
00645                                     frame->header.length,
00646                                     s_soft_bits,
00647                                     k_harq_soft_buffer_size,
00648                                     &soft_len);
00649
00650   const rx_harq_decode_params_t params = {
00651     .soft_bits          = s_soft_bits,
00652     .soft_len           = soft_len,
00653     .expected_output_len = internal_fec_decoded_len(soft_len),
00654   };
00655
00656   const rx_err_t err = rx_harq_decode(&link->harq, &params, result->payload, &result->payload_len);
00657
00658   result->fec_decoded     = true;
00659   result->combining_count = rx_harq_get_retry_count(&link->harq);
00660
00661   if (err == k_rx_ok) {
00662     /* Decode success - send ACK */
00663     const rx_err_t ack_err = rx_spi_comm_send_ack(link->spi_handle, frame->header.sequence);
00664     if (ack_err != k_rx_ok) {
00665       rx_log_warn(s_tag, "ACK send failed (data decoded OK)");
00666     }
00667     (void)rx_harq_reset(&link->harq);
00668     return k_rx_ok;
00669   }
00670
00671   /* Decode failed - send NACK for retransmission */
00672   rx_log_debug(s_tag, "FEC decode failed, sending NACK");
00673   (void)rx_spi_comm_send_nack(link->spi_handle, frame->header.sequence, k_frame_flag_soft_nack);
00674   return k_rx_err_protocol_error;
```

```
00675 }
00676
00677 /**
00678  * @brief Process received frame without FEC (passthrough mode)
00679  *
00680  * @details
00681  * Copies payload directly to result buffer and sends ACK if required
00682  * by frame flags. No FEC decoding or Chase Combining applied.
00683  *
00684  *
00685  */
00686 RX_STATIC_TESTABLE rx_err_t internal_receive_passthrough(rx_spi_link_t*                link,
00687                                                          const rx_frame_t*             frame,
00688                                                          rx_spi_link_receive_result_t* result)
00689 {
00690   /* No FEC: passthrough (copy payload directly). */
00691   if (frame->header.length > 0 && frame->header.length <= k_harq_max_payload) {
00692     for (uint16_t i = 0; i < frame->header.length; i++) {
00693       result->payload[i] = frame->payload[i];
00694     }
00695     result->payload_len = frame->header.length;
00696   } else if (frame->header.length > k_harq_max_payload) {
00697     /* Defensive: bound payload_len to buffer capacity to prevent out-of-bounds reads */
00698     rx_log_warn(s_tag, "Passthrough payload too large, truncating");
00699     for (uint32_t i = 0; i < k_harq_max_payload; i++) {
00700       result->payload[i] = frame->payload[i];
00701     }
00702     result->payload_len = k_harq_max_payload;
00703   } else {
00704     /* Zero-length payload */
00705     result->payload_len = 0;
00706   }
00707   result->fec_decoded     = false;
00708   result->combining_count = 1;
00709
00710   /* Send ACK if the frame requires it */
00711   if ((frame->header.flags & k_frame_flag_requires_ack) != 0) {
00712     const rx_err_t ack_err = rx_spi_comm_send_ack(link->spi_handle, frame->header.sequence);
00713     if (ack_err != k_rx_ok) {
00714       rx_log_warn(s_tag, "ACK send failed (passthrough)");
00715     }
00716   }
00717
00718   return k_rx_ok;
00719 }
00720
00721 /**
00722  * @brief Receive and decode a frame from SPI with HARQ
00723  * @see rx_spi_link.h for full documentation
00724  */
00725 rx_err_t
00726 rx_spi_link_receive(rx_spi_link_t* link, rx_spi_link_receive_result_t* result, uint32_t timeout_ms)
00727 {
00728   /* Pre-condition 1: NULL checks */
00729   if (link == nullptr || result == nullptr) {
00730     return k_rx_err_invalid_arg;
00731   }
00732
00733   /* Pre-condition 2: State check */
00734   if (!link->initialized) {
00735     return k_rx_err_invalid_state;
00736   }
00737
00738   /* Receive raw frame from SPI transport */
00739   rx_frame_t frame = {};
00740   *result          = (rx_spi_link_receive_result_t){};
00741
00742   const rx_err_t recv_err = rx_spi_comm_receive(link->spi_handle, &frame, timeout_ms);
00743   if (recv_err != k_rx_ok) {
00744     return recv_err; /* Timeout or error */
00745   }
00746
00747   /* Handle ACK/NACK control frames internally (not data for application) */
00748   if (frame.header.type == k_frame_type_ack || frame.header.type == k_frame_type_nack) {
00749     /* Control frame - not application data */
00750     return k_rx_err_no_data; /* Signal "no data" to caller */
00751   }
00752
00753   /* Handle PING/PONG/RESET (already handled by rx_spi_comm internally) */
00754   const bool is_ping      = (bool)(frame.header.type == k_frame_type_ping);
00755   const bool is_pong      = (bool)(frame.header.type == k_frame_type_pong);
00756   const bool is_reset     = (bool)(frame.header.type == k_frame_type_reset);
00757   const bool is_reset_ack = (bool)(frame.header.type == k_frame_type_reset_ack);
00758   if ((bool)((int)is_ping | (int)is_pong | (int)is_reset | (int)is_reset_ack)) {
00759     return k_rx_err_no_data; /* Not application data */
00760   }
00761
```

```
00762   /* Data frame received - populate common metadata */
00763   result->sequence      = frame.header.sequence;
00764   result->frame_type    = frame.header.type;
00765   result->is_retransmit = (bool)((frame.header.flags & k_frame_flag_retransmit) != 0);
00766
00767   /* Check if FEC decoding is needed */
00768   const bool has_fec_flag = (bool)((frame.header.flags & k_frame_flag_fec_enabled) != 0);
00769
00770   const bool fec_enabled_flag = link->fec_enabled;
00771   const bool fec_frame_flag   = has_fec_flag;
00772   const bool has_payload      = (bool)(frame.header.length > 0);
00773   if ((bool)((int)fec_enabled_flag & (int)fec_frame_flag & (int)has_payload)) {
00774     /* FEC decode path: HARQ with Chase Combining */
00775     return internal_receive_fec_decode(link, &frame, result);
00776   }
00777
00778   /* No FEC: passthrough (copy payload directly) */
00779   return internal_receive_passthrough(link, &frame, result);
00780 }
00781
00782 /*
      ============================================================================
00783  * State Management Implementation
00784  *
      ============================================================================
      */
00785
00786 /**
00787  * @brief Reset SPI link layer for new session
00788  * @see rx_spi_link.h for full documentation
00789  */
00790 rx_err_t rx_spi_link_reset(rx_spi_link_t* link)
00791 {
00792   /* Pre-condition 1 */
00793   if (link == nullptr) {
00794     return k_rx_err_invalid_arg;
00795   }
00796
00797   /* Pre-condition 2 */
00798   if (!link->initialized) {
00799     return k_rx_err_invalid_state;
00800   }
00801
00802   /* Reset HARQ (clears combiner and retry count). Only fails if harq is
00803    * null/uninitialized -- both prevented by link->initialized check above. */
00804   (void)(rx_harq_reset(&link->harq));
00805
00806   /* Post-condition: return to idle */
00807   link->state = k_spi_link_state_idle;
00808
00809   rx_log_debug(s_tag, "Link reset");
00810   return k_rx_ok;
00811 }
00812
00813 /**
00814  * @brief Get current SPI link state
00815  * @see rx_spi_link.h for full documentation
00816  */
00817 rx_spi_link_state_t rx_spi_link_get_state(const rx_spi_link_t* link)
00818 {
00819   if (link == nullptr || !link->initialized) {
00820     return k_spi_link_state_error;
00821   }
00822   return link->state;
00823 }
00824
00825 /**
00826  * @brief Check if FEC is enabled on this link
00827  * @see rx_spi_link.h for full documentation
00828  */
00829 bool rx_spi_link_fec_enabled(const rx_spi_link_t* link)
00830 {
00831   if (link == nullptr || !link->initialized) {
00832     return false;
00833   }
00834   return link->fec_enabled;
00835 }
```